

UNIVERSITATEA BABEȘ-BOLYAI  
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ  
SPECIALIZAREA INFORMATICĂ

## **LUCRARE DE LICENȚĂ**

**Învățare prin Întărire inițializată prin  
imitație pentru Luarea Deciziilor  
Strategice în jocul Catan**

**Coordonator științific  
Prof. dr. Czibula Gabriela**

**Student  
Turculeț Sorin-Nicolae**

BABEȘ-BOLYAI UNIVERSITY  
FACULTY OF MATHEMATICS AND COMPUTER  
SCIENCE  
SPECIALIZATION COMPUTER SCIENCE

## **DIPLOMA THESIS**

# **Imitation-Initialized Reinforcement Learning for Strategic Decision-Making in Settlers of Catan**

**Scientific supervisor**  
**Prof. dr. Czibula Gabriela**

**Student**  
**Turculeț Sorin-Nicolae**

## Abstract

The subject of this thesis is the application of Reinforcement Learning (RL) to the complex, multi-agent board game *Settlers of Catan*. Specifically, this work explores the impact of expert-guided learning on training autonomous agents within this highly stochastic and partially observable environment.

The thesis is structured into four chapters. The first chapter introduces the core mechanics and rules of *Settlers of Catan*, alongside the fundamental concepts of reinforcement learning. The second chapter details the environment modeling using the *Catanatron* framework and formally defines the reinforcement learning algorithms and methods applied to the problem. The third chapter presents and analyzes the experimental results, evaluating the agent's performance and learning progression. Finally, the fourth chapter outlines the software solution developed to demonstrate the trained agent's capabilities in live gameplay against human users.

The primary elements of originality in this thesis lie in the systematic evaluation and refinement of multiple policy optimization architectures, specifically expert-guided reinforcement learning and imitation learning, within the highly stochastic domain of *Settlers of Catan*. Rather than merely relying on custom reward shaping, this work isolates the structural limitations of heuristic guidance and demonstrates how decoupling expert knowledge through behavioral cloning successfully breaks established performance ceilings, ultimately achieving a peak win rate of 69.6% against a baseline heuristic. This research also identifies the “paradox of teacher quality,” providing empirical proof that pre-training a reactive agent on a strategically superior multi-ply search algorithm yields worse final performance than training on a simpler 1-ply heuristic due to unobserved intent. A further original contribution consists of the refinement of a dedicated software application that allows human users to play against the trained agent, providing a practical demonstration of the research outcomes.

This work is the result of my own activity. I have neither given nor received unauthorized assistance on this work.

Turculeț Sorin-Nicolae

Cluj-Napoca  
02-07-2026

# Contents

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| <b>List of Figures</b>                                                        | <b>2</b>  |
| <b>List of Tables</b>                                                         | <b>3</b>  |
| <b>List of publications</b>                                                   | <b>5</b>  |
| <b>Introduction</b>                                                           | <b>6</b>  |
| <b>1 Background</b>                                                           | <b>8</b>  |
| 1.1 The Game of Settlers of Catan . . . . .                                   | 8         |
| 1.2 Reinforcement Learning . . . . .                                          | 11        |
| 1.2.1 Fundamental Concepts of Reinforcement Learning . . . . .                | 11        |
| 1.2.2 Reinforcement Learning in Games . . . . .                               | 12        |
| 1.2.3 Imitation Learning . . . . .                                            | 12        |
| 1.3 Literature Review on Artificial Intelligence in Settlers of Catan . . . . | 13        |
| <b>2 An Approach Based on Reinforcement Learning</b>                          | <b>14</b> |
| 2.1 Problem Definition and Theoretical Challenges . . . . .                   | 15        |
| 2.2 Environment Modeling . . . . .                                            | 16        |
| 2.2.1 The Catanatron Environment . . . . .                                    | 16        |
| 2.2.2 Observation Space . . . . .                                             | 16        |
| 2.2.3 Action Space . . . . .                                                  | 19        |
| 2.2.4 Reward Functions . . . . .                                              | 20        |
| 2.2.5 Environment Complexity . . . . .                                        | 21        |
| 2.3 Reinforcement Learning Approach . . . . .                                 | 22        |
| 2.3.1 Theoretical Limitations of Value-Based Methods . . . . .                | 22        |
| 2.3.2 Proximal Policy Optimization . . . . .                                  | 23        |
| 2.3.3 Maskable PPO . . . . .                                                  | 25        |
| 2.3.4 Expert-Guided Reinforcement Learning . . . . .                          | 25        |
| 2.3.5 Imitation Learning Initialization (Behavioral Cloning) . . . . .        | 26        |
| 2.4 Training Setup . . . . .                                                  | 27        |
| 2.5 Baseline Opponent . . . . .                                               | 27        |
| <b>3 Experimental Results and Discussion</b>                                  | <b>29</b> |
| 3.1 Pure-PPO Dynamics: Establishing the Baseline . . . . .                    | 30        |
| 3.2 Direct Comparison: Pure-PPO vs. EG-PPO . . . . .                          | 31        |
| 3.3 Expert-Guided PPO: Breaking the Ceiling . . . . .                         | 32        |
| 3.4 Behavioral Cloning and Fine-Tuning (BC-PPO) . . . . .                     | 33        |
| 3.5 The Impact of Teacher Quality on Imitation Learning . . . . .             | 34        |

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| 3.5.1    | The Paradox of the Superior Teacher . . . . .          | 35        |
| 3.5.2    | Strategic Divergence . . . . .                         | 35        |
| 3.6      | Discussion . . . . .                                   | 36        |
| 3.6.1    | Comparison to Related Work . . . . .                   | 36        |
| 3.6.2    | Answers to Research Questions . . . . .                | 37        |
| 3.7      | Future Research Directions . . . . .                   | 38        |
| <b>4</b> | <b>System Architecture and Application Deployment</b>  | <b>39</b> |
| 4.1      | Analysis of the Baseline System Architecture . . . . . | 40        |
| 4.2      | Implementation . . . . .                               | 42        |
| 4.3      | Testing . . . . .                                      | 45        |
| 4.4      | User Manual . . . . .                                  | 45        |
| 4.5      | Conclusions and Future Enhancements . . . . .          | 46        |
|          | <b>Conclusions</b>                                     | <b>49</b> |
|          | <b>Acknowledgements</b>                                | <b>50</b> |

# List of Figures

|     |                                                                                                                                                                                   |    |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | A physical game of <i>Settlers of Catan</i> in progress, illustrating a real-life board configuration with players. Reproduced from [Cat95]. . . . .                              | 8  |
| 1.2 | Visualization of the probability distribution for rolling two six-sided dice, showing the approximate decimal values for each possible sum.                                       | 9  |
| 1.3 | The standard Reinforcement Learning interaction loop between Agent and Environment. Adapted from [SB18]. . . . .                                                                  | 11 |
| 2.1 | Tile identifiers and resource distribution in the Catan board. Reproduced from [Col23a]. . . . .                                                                                  | 17 |
| 2.2 | Node identifiers corresponding to settlement and city locations. Reproduced from [Col23a]. . . . .                                                                                | 18 |
| 4.1 | The baseline four-tier logical architecture of the web application. . . .                                                                                                         | 41 |
| 4.2 | The relational database schema demonstrating the mapping between authenticated users and persistent game sessions. . . . .                                                        | 44 |
| 4.3 | Comparison of the presentation tier assets. Left: The original circuit-board tiles (Reproduced from [Col23a]). Right: The newly integrated, AI-generated SVG biome tiles. . . . . | 45 |
| 4.4 | The user’s game history dashboard, listing prior sessions linked to the authenticated account. . . . .                                                                            | 46 |
| 4.5 | Game setup screen demonstrating the dynamic AI opponent selection.                                                                                                                | 47 |
| 4.6 | The main gameplay interface featuring the updated SVG board and real-time interaction. . . . .                                                                                    | 47 |

# List of Tables

|     |                                                                                                                                                                                                                               |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Resource costs for building and development in <i>Settlers of Catan</i> . . . .                                                                                                                                               | 9  |
| 2.1 | Relative strength of several built-in Catanatron agents in 1v1 games. Results reproduced from the Catanatron documentation [ <a href="#">Col23b</a> ]. . .                                                                    | 28 |
| 3.1 | Representative Pure-PPO runs at 10M steps, isolating different experimental axes. . . . .                                                                                                                                     | 30 |
| 3.2 | Ablation study results across 500 evaluation games using a sparse, terminal-only reward function. . . . .                                                                                                                     | 31 |
| 3.3 | Progression of EG-PPO agents evaluated over 500 games against the heuristic baseline. The integration of expert guidance immediately breaks the 10% Pure-PPO ceiling and successfully scales with additional compute. . . . . | 32 |
| 3.4 | Progression of BC-PPO agents compared to the EG-PPO ceiling. The BC-PPO agents were pre-trained on 100,000 expert games before unconstrained fine-tuning. . . . .                                                             | 34 |
| 3.5 | Win rates of BC-pretrained agents evaluated against VFP and Alpha-Beta (depth=2). All evaluations consist of 500 head-to-head games. .                                                                                        | 35 |
| 3.6 | Play style statistics for BC-pretrained agents across evaluation opponents. Road = longest road indicator, Army = largest army indicator, Dev VP = average VP from development cards. . . . .                                 | 36 |

# List of publications

- [SN26] **Turculeț Sorin-Nicolae.** *Imitation-Initialized Reinforcement Learning for Strategic Decision-Making in Settlers of Catan.* Studia Babeș-Bolyai Informatica, 2026, submitted.

# Introduction

Learning **rarely** begins from scratch.

Human decision-making builds upon knowledge, heuristics, and experience acquired over time, much like how scientific progress rests on previously established foundations refined through experimentation. In the realm of Artificial Intelligence, games have long served as the ultimate proving ground for evaluating these cumulative learning processes. While Reinforcement Learning (RL) has recently achieved superhuman performance in perfect-information games such as Chess and Go by exploring vast decision trees with perfect clarity, many approaches still rely on minimal prior knowledge. In complex stochastic and adversarial settings, this assumption that an agent must learn entirely from scratch often leads to slow convergence, inefficient exploration, and unstable training dynamics.

The board game *Settlers of Catan* provides a controlled yet challenging environment for studying these phenomena, combining stochastic resource generation, adversarial interaction, and long-term strategic planning. Even in a simplified 1v1 configuration, players must balance immediate resource gains with long-term expansion and positional advantages, making the learning task highly non-trivial. Consequently, this thesis investigates whether incorporating expert knowledge as guidance can not only accelerate but also fundamentally improve the learning process in a competitive stochastic environment, using *Settlers of Catan* as a primary case study.

## Original Contributions

To tackle these challenges, this thesis introduces four main contributions. First, we evaluate Expert-Guided PPO (EG-PPO), showing that giving the agent step-by-step rewards based on expert rules helps it learn much faster than starting completely from scratch. Second, we build a two-step Imitation Learning pipeline (BC-PPO). By pre-training the agent on expert games before letting it learn on its own, we break past the limits of standard reward shaping. Third, we investigate how the quality of the expert teacher affects the AI, specifically comparing the outcomes of training an agent using a complex, deep-searching expert versus a simple, predictable expert. Finally, we refined a live web application. Using WebSockets and a dynamic Stable-Baselines3 model registry, this platform allows human players to test their skills against the trained AI in real-time.

## Structure of the Thesis

The remainder of this thesis is structured as follows. Chapter 1 introduces the core mechanics of *Settlers of Catan* alongside the fundamental concepts of Reinforcement Learning and Imitation Learning. The environment modeling and our proposed policy optimization methodologies are formally outlined in Chapter 2.

Chapter 3 is concerned with presenting a rigorous analysis of the training dynamics and evaluating the agent’s performance, while a detailed software solution transitioning the offline models into a deployed web application is presented in Chapter 4.

## **Declaration of Generative AI and AI-assisted technologies in the writing process**

During the preparation of this work the author used Generative AI tools in order to assist with initial literature research, generate parts of the application’s source code, and refine the formatting and formulation of the academic text. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the thesis.

## **Code Availability**

The complete source code developed for this thesis, including the custom reinforcement learning training pipelines, the environment modifications, and the dynamic web application architecture, is open source and publicly available on GitHub at: <https://github.com/sorinturculet/catanatron/tree/pr-297>.

# Chapter 1

## Background

The purpose of this chapter is to introduce the game of *Settlers of Catan* and some of the main concepts used in this thesis, creating an underlying basis for the sequential decision-making problem and its place in the research world. The chapter is structured as follows.

### 1.1 The Game of Settlers of Catan

*Settlers of Catan* is a multiplayer strategy board game designed by Klaus Teuber and first released in 1995. Since its introduction, the game has become one of the most popular modern board games, with millions of copies sold worldwide and translations into numerous languages [Cat95]. Its success has led to the development of numerous expansions, digital adaptations, and spin-offs. Many of these expansions introduce additional mechanics and rules, increasing the strategic depth and variability of the gameplay. Figure 1.1 illustrates an example configuration of the board during a physical game, highlighting the hexagonal terrain tiles, player constructions, and resource cards.



Figure 1.1: A physical game of *Settlers of Catan* in progress, illustrating a real-life board configuration with players. Reproduced from [Cat95].

The game is typically played by three or four players on a hexagonal board representing an island composed of different terrain tiles. Each hexagonal terrain tile

produces a specific resource, such as wood, brick, wheat, sheep, or ore. These resources form the basis of the game's economic system, allowing players to construct roads, settlements, and cities. Table 1.1 summarizes the resource costs required for each construction.

| Item             | Wood | Brick | Sheep | Wheat | Ore |
|------------------|------|-------|-------|-------|-----|
| Road             | 1    | 1     | 0     | 0     | 0   |
| Settlement       | 1    | 1     | 1     | 1     | 0   |
| City             | 0    | 0     | 0     | 2     | 3   |
| Development Card | 0    | 0     | 1     | 1     | 1   |

Table 1.1: Resource costs for building and development in *Settlers of Catan*.

Each terrain tile is associated with a number token between 2 and 12, except for the number 7. Because resource production is tied to rolling two six-sided dice, the probability of a tile producing resources follows a triangular distribution. The probability  $P$  of rolling a specific sum  $x$  is defined as:

$$P(X = x) = \frac{6 - |x - 7|}{36}, \quad \text{for } x \in \{2, 3, \dots, 12\}$$

The resulting probability values for each possible sum from 2 to 12 are visualized in Figure 1.2. This distribution clearly shows that the most frequent outcome is a roll of 7 (the peak), which corresponds to a probability of approximately 0.167. Conversely, the least likely outcomes are rolls of 2 and 12, each with a probability of approximately 0.028.

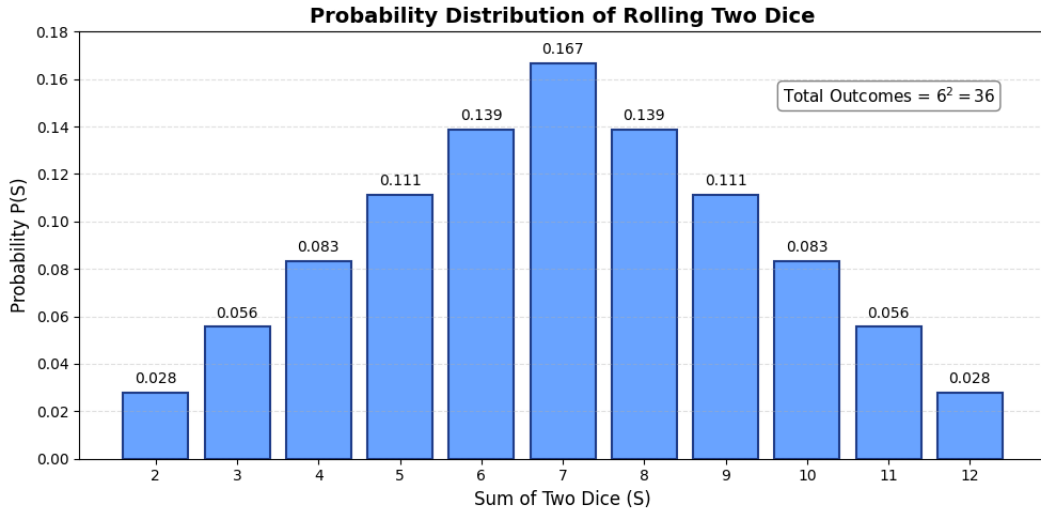


Figure 1.2: Visualization of the probability distribution for rolling two six-sided dice, showing the approximate decimal values for each possible sum.

At the beginning of the game, players place their initial pieces, consisting of two settlements and two roads. The placement order is determined randomly, typically by having each player roll the dice. The first settlement is placed following this order, while the second settlement is placed in reverse order, ensuring a more balanced initial distribution of resources among players. An important rule governing

settlement placement is the distance rule, which requires that settlements must be separated by at least two edges on the board.

As a result, in games with four players, certain configurations of the initial placements may prevent the first player, who places settlements in the first and eighth turns, from selecting a second settlement that is adjacent to three different resource tiles. This situation may create an initial disadvantage, which is **partially** compensated by the fact that the first player typically selects the most favorable location on the board for their first settlement.

The game proceeds in turns, with each turn following a fixed sequence of actions:

1. The active player rolls two dice.
2. Depending on the result of the dice roll, one of the following situations occurs:
  - (a) If the result is different from 7, resources are produced. For each terrain tile whose number token matches the rolled value, players with settlements adjacent to that tile receive one resource card of the corresponding type. Cities produce two resource cards instead of one.
  - (b) If the result of the dice roll is 7, a special sequence is triggered. First, every player holding more than seven resource cards must discard half of them (rounded down). Then, the active player moves the robber to any terrain tile of their choice. The robber blocks resource production from that tile while it remains there. Additionally, the active player may choose one player with a settlement or city adjacent to that tile and randomly steal one of their resource cards.
3. The player can use resources to buy development cards, which are random special cards that provide various bonuses, or to build and expand. During the same turn, the player may also trade with any other player. Additionally, players can trade with the bank at a cost of 4:1 resources, or at ratios of 3:1 or 2:1 if they have a settlement adjacent to a port located on the edge of the island.

The game ends when a player reaches 10 victory points. Victory points are obtained according to the following rules:

- 1 victory point for each settlement.
- 2 victory points for each city.
- 2 victory points for having the largest army.
- 2 victory points for having the longest road.
- 1 victory point for each victory point development card.

## 1.2 Reinforcement Learning

### 1.2.1 Fundamental Concepts of Reinforcement Learning

Reinforcement learning (RL) is a machine learning paradigm in which an agent learns to make decisions through interaction with an environment. Unlike supervised learning, where models are trained on labeled datasets, reinforcement learning relies on trial-and-error exploration combined with feedback in the form of rewards [SB18, KLM96]. The objective of the agent is to learn a strategy that maximizes the cumulative reward obtained over time.

In a reinforcement learning setting, the agent observes the current state of the environment and selects an action according to a policy. After executing the action, the environment transitions to a new state and returns a reward signal indicating the quality of the chosen action. Through repeated interactions with the environment, the agent gradually improves its policy in order to achieve higher long-term rewards.

As illustrated in Figure 1.3, this standard reinforcement learning loop consists of continuous, cyclical interaction between the Agent and the Environment.

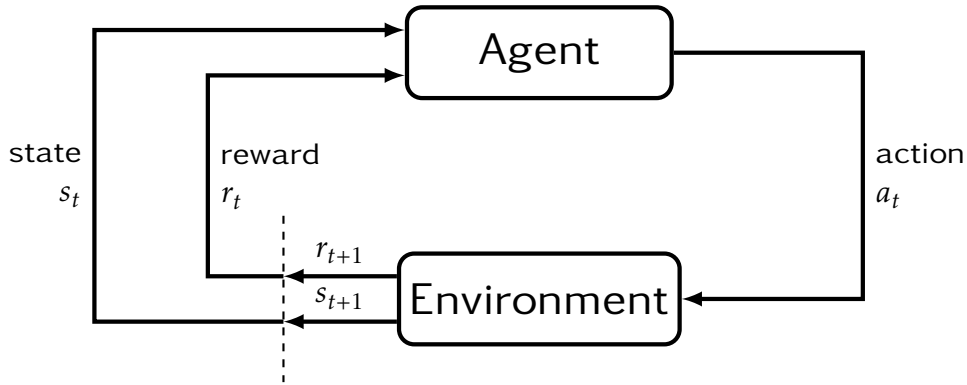


Figure 1.3: The standard Reinforcement Learning interaction loop between Agent and Environment. Adapted from [SB18].

Reinforcement learning problems are commonly modeled using a Markov Decision Process (MDP). An MDP is formally defined as the tuple  $(S, A, P, R, \gamma)$ , where  $S$  represents the set of possible states,  $A$  denotes the set of actions available to the agent,  $P(s' | s, a)$  describes the probability of transitioning from state  $s$  to state  $s'$  after taking action  $a$ ,  $R(s, a)$  represents the reward function, and  $\gamma \in [0, 1)$  is the discount factor that determines the relative importance of future rewards [SB18].

The goal of the agent is to maximize the expected cumulative reward obtained during its interaction with the environment. This cumulative reward, commonly referred to as the return, is defined as

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

where  $R_{t+k+1}$  denotes the reward received at time step  $t + k + 1$ , and  $\gamma$  controls how strongly future rewards are discounted compared to immediate rewards.

To evaluate the quality of decisions, reinforcement learning methods rely on value functions. The state-value function  $V^\pi(s)$  represents the expected return ob-

tained when starting from state  $s$  and following a policy  $\pi$ . Similarly, the action-value function  $Q^\pi(s, a)$  represents the expected return obtained when taking action  $a$  in state  $s$  and subsequently following the same policy [SB18].

Finally, reinforcement learning relies on the principle that decisions should be evaluated based on their long-term consequences. This idea is captured by the Bellman optimality equation, which states that the value of a state under an optimal policy must equal the expected return for the best action from that state:

$$V^*(s) = \max_a \sum_{s', r} P(s', r | s, a) [r + \gamma V^*(s')]$$

This principle forms the theoretical foundation of many reinforcement learning algorithms.

### 1.2.2 Reinforcement Learning in Games

Games have long served as important benchmarks for artificial intelligence (AI) research. They provide well-defined environments with clear rules, measurable outcomes, and often complex strategic interactions between agents. Because of these properties, games represent a suitable domain for evaluating reinforcement learning algorithms and measuring the progress of artificial intelligence systems.

In recent years, reinforcement learning has achieved remarkable success in a variety of game environments. One notable example is the Atari 2600 benchmark, where deep reinforcement learning agents learned to play dozens of video games directly from raw pixel input using a combination of neural networks and Q-learning techniques [M<sup>+</sup>15].

Reinforcement learning has also achieved superhuman performance in complex board games. Systems such as AlphaGo and AlphaZero successfully learned to play games such as Go, Chess, and Shogi at levels exceeding human experts by combining deep neural networks with reinforcement learning and self-play [SHM<sup>+</sup>16, S<sup>+</sup>18].

Despite these successes, many strategic board games remain challenging for reinforcement learning due to factors such as stochasticity, large action spaces, and interactions between multiple agents. Games that involve randomness or negotiation between players introduce additional complexity compared to perfect-information games such as chess or Go.

### 1.2.3 Imitation Learning

A fundamental limitation of standard reinforcement learning in complex, high-variance environments is the inefficiency of random exploration. In games with massive state-action spaces and sparse, delayed rewards, such as *Settlers of Catan*, where the only definitive reward is winning the game after hundreds of steps, an agent relying purely on random trial-and-error may never discover a winning strategy.

To overcome this sparse reward problem, researchers often employ Imitation Learning (IL). This paradigm accelerates the learning process by leveraging prior domain knowledge or expert demonstrations to guide and “warm-start” the RL agent’s policy before it begins independent exploration [SHM<sup>+</sup>16].

In this manner, fundamental game strategies can be inherited through the imitation of a capable, though not necessarily optimal, expert. For example, rather than relying on random exploration to understand the value of resources, the agent learns to prioritize high-yield intersections during the game’s initial placement phase by observing the expert’s decisions.

### 1.3 Literature Review on Artificial Intelligence in Settlers of Catan

While reinforcement learning has seen significant success in perfect-information games, applying AI to *Settlers of Catan* introduces unique challenges due to its stochasticity, imperfect information (hidden cards), and complex multi-agent interactions (trading).

One of the earliest reinforcement learning attempts at *Settlers of Catan* was Pfeiffer’s work [Pfe04], which combined hierarchical RL with hand-crafted heuristics and trained through self-play. Instead of treating Catan as one big decision problem, Pfeiffer split the game into separate sub-behaviors, such as settlement placement and trading, and learned a policy for each one. Because deep neural networks were not yet practical, model trees (a tree-based regression method) were used as the function approximator. The author tested four variants of this hierarchical setup and found that the ones mixing learning with built-in domain knowledge clearly outperformed pure self-play, producing agents that could challenge and sometimes beat human players.

Beyond reinforcement learning, a parallel line of work pursued search-based methods to handle Catan’s enormous branching factor. Szita et al. [SCS10] successfully applied Monte Carlo Tree Search (MCTS) to Catan, demonstrating that simulating random future rollouts could yield strong strategic decisions. However, deep search trees quickly become computationally intractable in this domain without aggressive pruning, because every move opens a huge fan-out of dice and trade outcomes.

Whelan et al. [WMB23] investigated the efficacy of Deep Q-Networks (DQN), Double DQN, and Dueling Networks. However, to make the massive action space computationally tractable for these value-based methods, their environment required drastic simplifications: they were forced to remove inter-player trading entirely, lower the victory condition to only 6 points, and restrict the matches to a 1v1 setting. Even with these heavy constraints, their best-performing model struggled to find a winning strategy, achieving a maximum win rate of only 3% against a baseline Alpha-Beta pruning bot.

Building upon this context, this thesis also adopts a strict two-player (1v1) configuration. By reducing the game from its traditional three- or four-player format, the complex, non-deterministic dynamics of multi-agent negotiation, most notably inter-player trading, are entirely eliminated. This simplification is necessary to manage the state-action space complexity while still maintaining the standard 10 victory point win condition. By implementing Proximal Policy Optimization (PPO) augmented with imitation learning initialization, this work aims to evaluate the potential for faster learning convergence and stronger strategic performance compared to prior value-based methods.

## Chapter 2

# An Approach Based on Reinforcement Learning for Settlers of Catan

This chapter presents the reinforcement learning approach used to train an agent capable of playing the board game *Settlers of Catan*. While the previous chapter described reinforcement learning as a general concept, in the following pages, *Settlers of Catan* is formulated as a reinforcement problem.

Training reinforcement learning agents in complex environments such as Catan presents several challenges. The game contains a large number of possible states and actions, stochastic elements introduced by dice rolls, and interactions between multiple players. These characteristics significantly increase the complexity of the learning process and make efficient exploration difficult. As a result, naive reinforcement learning approaches often require a large number of training episodes before meaningful strategies emerge.

To address these challenges, this work employs a policy-gradient reinforcement learning method based on *Proximal Policy Optimization (PPO)* combined with *Maskable PPO*, which allows the agent to consider only valid actions at each decision step. In addition, an expert-guided learning mechanism is introduced in order to guide exploration during training by incorporating feedback from a heuristic player. This approach helps the agent focus on strategically meaningful actions while still allowing it to learn policies that may surpass the expert strategy.

To evaluate this methodology and address the challenges of the *Settlers of Catan* environment, this work is guided by the following three core research questions:

- RQ1. Mitigating Sparse Rewards:** How does the integration of heuristic expert guidance (EG-PPO) overcome the credit assignment bottleneck compared to training an agent entirely from scratch (Pure-PPO)?
- RQ2. The Impact of Imitation Learning:** To what extent does initializing the policy network with expert demonstrations (BC-PPO) improve sample efficiency and final win rates compared to relying solely on online reward shaping?
- RQ3. Teacher Quality Scaling:** How does the strategic depth of the expert demonstrator (e.g., a multi-ply Alpha-Beta search versus a 1-ply Value Function) affect the ultimate performance ceiling of the fine-tuned reinforcement learning agent?

## 2.1 Problem Definition and Theoretical Challenges

The fundamental problem addressed in this thesis is the formulation of *Settlers of Catan* as a sequential decision-making process, enabling an artificial agent to learn winning strategies through reinforcement learning and expert guidance.

Although *Settlers of Catan* is a board game, it serves as a highly effective proxy for real-world systems involving decentralized decision-making under uncertainty. The challenges encountered in this domain closely mirror those found in practical applications:

- **Resource Allocation and Logistics:** The prioritization of limited resources and long-term planning strategies directly corresponds to optimization problems in supply chains and industrial logistics.
- **Multi-Agent Coordination:** Strategic interaction between players reflects coordination challenges in distributed systems, where agents must operate under partial information and competing objectives.
- **Stochastic Decision-Making:** The probabilistic nature of resource generation introduces uncertainty similar to financial markets, smart grids, and autonomous planning systems.

To formalize this, the game is modeled as a discrete-time Markov Decision Process (MDP). At each discrete time step  $t$ , the agent observes the environment's state  $s_t \in \mathcal{S}$ , which encompasses the board layout, current resource allocations, and opponent data, and selects an action  $a_t \in \mathcal{A}$  from a set of legally permissible moves according to its policy  $\pi$ . Following the execution of this action, the environment transitions to a subsequent state  $s_{t+1}$  based on the game's stochastic transition dynamics and yields a scalar reward  $r_t \in \mathcal{R}$ .

From a theoretical reinforcement learning perspective, solving this MDP is exceptionally difficult due to four inherent characteristics of the environment:

1. **Stochasticity:** Resource production depends on dice rolls, requiring the agent to reason probabilistically about future resource availability.
2. **Sparse and Delayed Rewards:** A strong early-game decision (e.g., optimal settlement placement) may only yield a tangible victory point many turns later, creating a severe temporal credit assignment problem.
3. **Partial Observability:** Players do not have complete knowledge of opponents' private resources or hidden development cards.
4. **Adversarial Multi-Agent Dynamics:** The quality of an action depends not only on the static board state but also on the opponent's anticipated future decisions.

As established in the previous chapter, the unrestricted multi-player version of *Settlers of Catan* presents an intractable action space for pure learning algorithms. Therefore, to specifically isolate and evaluate the agent's ability to learn spatial expansion, resource management, and long-term planning, the problem is scoped to a two-player (1v1) competitive setting where inter-player trading is disabled. The game concludes when a player reaches the standard condition of 10 Victory Points.

Because the state space of *Settlers of Catan* is astronomically large (as detailed in Section 2.2.5), traditional value-based methods, which attempt to compute the expected return for every individual state-action pair, become computationally unfeasible as the number of states tends toward infinity. Therefore, the problem dictates a policy-optimization approach. The agent’s ultimate objective is to utilize an actor network to directly parameterize and discover an optimal policy  $\pi^*(a_t|s_t)$  that maximizes the expected cumulative discounted reward. Navigating this massive combinatorial space, characterized by sparse and delayed feedback, further necessitates the integration of expert heuristics to guide the actor efficiently toward a winning strategy.

## 2.2 Environment Modeling

In order for an agent to learn to play *Settlers of Catan*, the game must be modeled as an environment with which the agent can interact. Within this environment, the agent observes the current state of the game, selects actions from the set of available moves, executes them, and receives feedback that reflects the consequences of its decisions. The environment is responsible for enforcing the rules of the game, preventing illegal actions, updating the game state after each move, and managing game mechanics such as resource exchanges and trades.

The following sections describe the main components of the environment used for training the agent. These include the implementation of the *Catanatron* environment, the representation of game states through the observation space, the definition of the action space available to the agent, the reward design used to guide the learning process, and the complexity of the resulting decision-making environment.

### 2.2.1 The Catanatron Environment

Rather than implementing a custom game simulator from scratch, this work uses the *Catanatron* environment [Col23a], which provides a fully functional implementation of the *Settlers of Catan* game together with support for reinforcement learning through an OpenAI Gymnasium interface. The environment implements the core rules of the game, including resource generation, construction actions, and trading between the player and the bank.

At the time of writing this thesis, some game features are not implemented in the framework, such as the restriction that prevents red number tokens from being placed adjacent to each other and direct trading between players. Nevertheless, even without these rules, the environment remains highly complex due to the large number of possible game states, the stochastic nature of dice rolls, and the wide variety of actions available to the agent. As a result, it provides a suitable and challenging environment for studying reinforcement learning approaches for playing *Settlers of Catan*.

### 2.2.2 Observation Space

The observation space represents the current state of the game in a form that can be processed by the reinforcement learning agent. In the *Catanatron* environment,

the observation is represented using a hybrid encoding that combines a spatial tensor describing the game board with a numerical feature vector containing global information about the match.

This mixed representation allows the model to capture both the spatial structure of the board and abstract aspects of the game state such as resource quantities, player scores, and strategic indicators.

### 2.2.2.1 Board Representation

The *Settlers of Catan* board consists of 19 hexagonal tiles, 54 intersection nodes where settlements and cities can be built, and 72 edges where roads may be placed. Each tile is associated with a resource type and a dice number that determines when resources are produced.

Figure 2.1 illustrates the tile indexing used by the environment. Each tile is assigned a unique identifier together with its corresponding resource type and dice number.

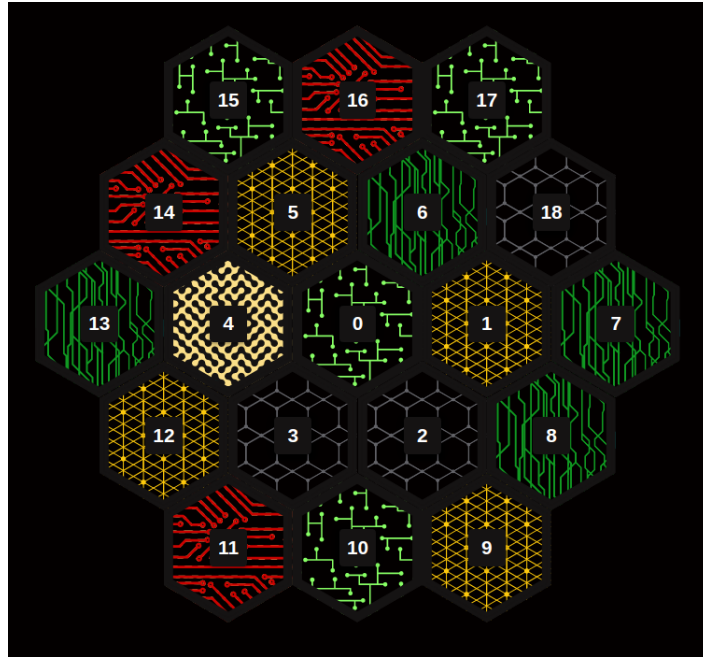


Figure 2.1: Tile identifiers and resource distribution in the Catan board. Reproduced from [Col23a].

The intersection nodes where constructions may be placed are also indexed. Figure 2.2 shows the numbering of the 54 nodes used by the environment.

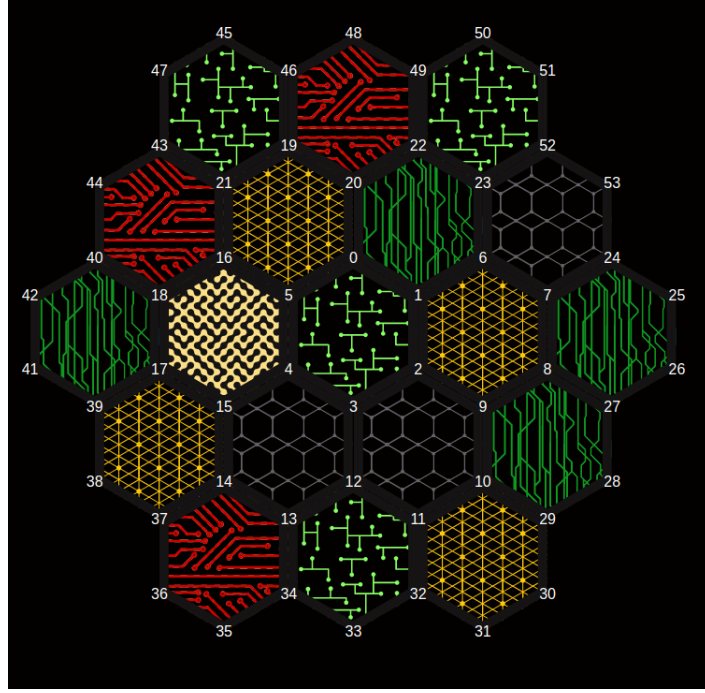


Figure 2.2: Node identifiers corresponding to settlement and city locations. Reproduced from [Col23a].

#### 2.2.2.2 Spatial Tensor Representation

To enable the use of convolutional neural networks, the hexagonal game board is projected onto a rectangular grid of size  $21 \times 11$ . This projection preserves adjacency relationships between tiles, nodes, and edges while allowing the board to be processed using standard convolutional operations.

Each observation contains a tensor composed of several feature planes, where each channel encodes a different aspect of the board state.

These channels include:

- Construction channels representing settlements, cities, and roads for both players.
- Resource channels encoding the type of resource associated with each tile.
- Dice probability channels reflecting the likelihood of resource production.
- A robber channel indicating the tile currently blocked by the robber.
- Port channels representing the location of specialized and generic trade ports.

This representation allows the neural network to capture spatial relationships between resources, constructions, and strategic locations on the board.

#### 2.2.2.3 Numerical Feature Vector

In addition to the spatial tensor, the observation also includes a numerical feature vector describing global properties of the game that cannot be easily encoded spatially.

These features include:

- Player victory points and game progress indicators.
- Resource cards held by the agent.
- Public information about the opponent's cards.
- Remaining resources and development cards available in the bank.
- Estimated resource production statistics for each player.
- Additional strategic indicators describing the expansion and city-building potential of each player.

#### 2.2.2.4 Hybrid Representation

The spatial tensor and numerical feature vector are processed jointly by the neural network used in the reinforcement learning algorithm. The convolutional layers extract spatial patterns from the board representation, while the numerical features provide global context about the game state.

Combining these two sources of information allows the agent to make decisions based on both local board configurations and overall strategic conditions of the game.

### 2.2.3 Action Space

The action space defines all the actions that the agent can perform during the game. In the Catanatron environment, the action space is modeled as a discrete space containing 290 possible actions, each corresponding to a specific move allowed by the rules of *Settlers of Catan*.

The actions are grouped into several categories reflecting the structure of the game:

- Dice roll (1 action), used to begin a player's turn.
- Robber placement (19 actions), corresponding to the possible tiles where the robber can be moved.
- Returning cards (1 action), triggered when a player must discard resources after a roll of seven.
- Building roads (72 actions), representing each possible edge of the board.
- Building settlements (54 actions), corresponding to the intersection nodes of the board.
- Upgrading settlements to cities (54 actions).
- Buying a development card (1 action).
- Playing development cards (27 actions), including knights, road building, year of plenty, and monopoly.

- Maritime trading with the bank (60 actions), covering all valid resource exchange combinations.
- Ending the turn (1 action).

Although the formal action space contains 290 actions, only a small subset is valid at any given state. For example, a player cannot build a settlement without the required resources or if no valid positions are available.

To address this issue, the environment uses *action masking*. A binary mask of size 290 indicates which actions are currently legal. Invalid actions are assigned zero probability before applying the softmax function in the policy network, ensuring that the agent selects only valid actions.

This mechanism is supported by the *Maskable PPO* algorithm, which extends the standard PPO framework to handle masked action spaces. In practice, although the full action space contains 290 actions, typically only between 1 and 20 actions are valid at any decision step, making the problem tractable for reinforcement learning.

## 2.2.4 Reward Functions

Designing the reward function proved to be one of the most important factors influencing the performance of the reinforcement learning agent. A key challenge arises from the structure of *Settlers of Catan*: games typically last between 200 and 400 decision steps, while the final outcome (win or loss) is observed only at the end of the episode. Such delayed rewards make the learning process difficult, as many intermediate actions do not immediately change the score but are essential for long-term success.

To address this and evaluate different training dynamics, two distinct reward functions were designed: a progress-based Dense VP reward and a behavior-guided Shaped reward.

### 1. Dense VP Reward

The foundational reward function ties the agent’s intermediate feedback strictly to its tangible progress toward winning the game. This function relies solely on changes in Victory Points (VP) and the terminal state:

$$r_t^{\text{VP}} = 0.1 \cdot \Delta VP_t + R_{\text{terminal},t}$$

where  $\Delta VP_t = VP_t - VP_{t-1}$  represents the change in victory points since the previous step. This includes positive changes for building structures or acquiring cards, as well as negative changes (e.g.,  $-0.1$ ) if the agent loses the Longest Road or Largest Army condition. The terminal reward  $R_{\text{terminal},t}$  is defined as  $+1.0$  for winning the game and  $-1.0$  for losing.

### 2. Shaped Environment Reward

To investigate whether explicitly guiding the agent’s sub-goals accelerates learning, an alternative shaped reward function was evaluated. This function combines the Dense VP reward with additional signals related to resource production and

construction actions. Increasing resource production is rewarded because it improves the agent's long-term ability to expand. Similarly, small bonuses are assigned when the agent builds roads or settlements, as these actions represent important strategic steps even when they do not immediately alter the VP score.

The shaped environment reward is formally expressed as:

$$r_t^{\text{shaped}} = 0.10 \cdot \Delta VP_t + 0.02 \cdot \Delta OWS_t + 0.04 \cdot \Delta BW_t + 0.10 \cdot \Delta S_t + 0.04 \cdot \Delta R_t + R_{\text{terminal},t}$$

where:

- $\Delta VP_t$  is the change in victory points.
- $\Delta OWS_t$  corresponds to the change in the production probability of ore, wheat, and sheep.
- $\Delta BW_t$  represents the change in the production probability of brick and wood.
- $\Delta S_t$  is a binary indicator (1 or 0) representing whether a settlement was built during step  $t$ .
- $\Delta R_t$  is a binary indicator (1 or 0) representing whether a road was built during step  $t$ .

It should be explicitly noted that the coefficients used in this shaped reward function are not optimal by any means. The rigorous optimization or hyperparameter tuning of these specific weightings falls outside the scope of this thesis. However, practical tests demonstrated that these values provided a sufficiently stable signal to guide the agent toward meaningful strategies during several experimental runs.

In both formulations, the coefficients were chosen so that intermediate rewards remain significantly smaller than the terminal reward. This ensures that the agent ultimately optimizes for winning the game rather than endlessly maximizing intermediate signals such as resource production or construction counts.

### 2.2.5 Environment Complexity

Although *Settlers of Catan* appears simpler than classical board games such as chess, it presents several challenges for reinforcement learning agents.

First, the board configuration is randomized at the beginning of every game. Different arrangements of resource tiles and number tokens create a large number of possible starting states. Previous analyses estimate that the number of valid tile and token configurations alone exceeds  $10^{27}$ , requiring the agent to learn strategies that generalize across many different board layouts rather than memorizing specific openings.

The overall state space becomes significantly larger when the full game state is considered. The environment must represent the placement of settlements and cities on the 54 nodes of the board, the placement of roads on 72 edges, the resources held by both players, the distribution of development cards, and the robber position. A rough estimate of the dimensionality of the state space can be expressed as  $10^{27} \times 5^{54} \times 3^{72} \times 20^{10} \times 10^8 \times 19 \approx 10^{120}$ , where the factors represent board configurations, building placements, road placements, resource distributions, development

card states, and robber positions. This illustrates the enormous combinatorial complexity of the environment.

In addition to the large state space, the game is stochastic due to dice rolls and development card draws, and it involves imperfect information since players cannot observe their opponent’s resources or cards. Each turn also consists of multiple sequential decisions such as building, trading, and ending the turn, which increases the effective branching factor of the game.

Together, the combination of stochastic dynamics, imperfect information, large state space, and multi-step decision structure makes *Settlers of Catan* a highly challenging environment for reinforcement learning.

## 2.3 Reinforcement Learning Approach

### 2.3.1 Theoretical Limitations of Value-Based Methods

Before detailing the specific algorithm utilized in this research, it is necessary to theoretically justify the selection of a policy-optimization approach over traditional value-based methods. The theoretical comparison between policy-gradient methods and value-function approximation has been explicitly studied in foundational reinforcement learning literature, such as the comparative analysis by McGovern et al. [MFPS06]. Their work emphasizes that neither family of methods is universally superior; rather, their algorithmic suitability depends entirely on the underlying structure of the environment.

Value-based methods, such as Q-learning and Deep Q-Networks (DQN), attempt to approximate the expected return function  $Q(s, a)$  for state-action pairs. The policy is then usually derived by selecting the action with the highest estimated value:

$$a = \arg \max_a Q(s, a)$$

While highly effective in smaller or more structured environments, these methods struggle in domains with large state-action spaces. *Settlers of Catan* is especially problematic for value-based algorithms because the value of an action is highly contextual. For example, building a road, buying a development card, or upgrading a settlement can be a strategically strong or weak move depending entirely on the current board layout, available resources, opponent positioning, and future expansion possibilities. Because value-based methods must explicitly evaluate these highly variable actions across an enormous combinatorial state space, they suffer from difficult function approximation, highly unstable value estimates, and poor sample efficiency.

This theoretical limitation explains the poor performance observed in prior literature attempting to apply DQN to *Settlers of Catan*, where agents failed to achieve meaningful win rates even under severely simplified rulesets.

In contrast, policy-based methods directly optimize the action distribution  $\pi_\theta(a|s)$ , bypassing the need to compute precise expected returns for every possible state-action pair. As shown in comparative literature [MFPS06], this allows policy-gradient methods to scale more effectively in large, high-variance domains. Policy optimization also supports stochastic policies, which are critical in environments where probabilistic reasoning and uncertainty are central components of the game.

To combine the benefits of both paradigms, modern algorithms utilize the Actor-Critic framework. The Actor represents the policy  $\pi_\theta(a|s)$ , while the Critic estimates the value function  $V(s)$ . This dual structure allows for more stable learning by reducing variance in policy updates while preserving the scalability of direct policy optimization. Because of these structural and theoretical advantages, this research exclusively employs policy-gradient Actor-Critic techniques.

### 2.3.2 Proximal Policy Optimization

Proximal Policy Optimization (PPO) is the core learning algorithm used to train the agent in this work. In reinforcement learning, a common problem is that an agent can easily overreact to a single good or bad experience. If it finds a strategy that works once, it might update its internal network so aggressively that it forgets other important strategies, a problem known as catastrophic forgetting. PPO is designed specifically to prevent this by enforcing a strict limit on how much the agent can change its policy during a single training step.

PPO uses an Actor-Critic architecture, meaning the agent relies on two separate neural networks working together:

- **The Actor (Policy Network):** This network observes the game board and decides which action to take. It outputs a probability distribution over all possible moves, denoted as  $\pi_\theta(a_t|s_t)$ .
- **The Critic (Value Network):** This network evaluates the game board and predicts how favorable the current situation is. It estimates the total future reward the agent can expect to receive from this state, denoted as  $V(s_t)$ .

**The Advantage Function.** To determine if a specific action was actually a strong move, PPO utilizes the Advantage function, denoted as  $\hat{A}_t$ . The Advantage compares the actual result of an action against what the Critic predicted would happen.

- If  $\hat{A}_t > 0$ , the action yielded a better outcome than expected. The agent should increase the probability of taking this action.
- If  $\hat{A}_t < 0$ , the action yielded a worse outcome than expected. The agent should decrease the probability of taking this action.

**The Probability Ratio.** To measure how much the agent is changing its behavior, the algorithm calculates the probability ratio:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)},$$

where:

- $r_t(\theta)$ : The probability ratio. It represents how much more (or less) likely the agent is to take this specific action now, compared to before the update.
- $\pi_\theta(a_t|s_t)$ : The probability of taking action  $a_t$  in state  $s_t$  under the **new** (updated) policy.
- $\pi_{\theta_{\text{old}}}(a_t|s_t)$ : The probability of taking action  $a_t$  in state  $s_t$  under the **old** (previous) policy.

**The Clipped Objective Function.** To ensure the agent does not update its strategy too drastically, PPO maximizes the following clipped objective function:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[ \min \left( r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right],$$

where:

- $L^{CLIP}(\theta)$ : The clipped objective function. This is the main mathematical goal the Actor seeks to maximize.
- $\mathbb{E}_t$ : The expected value, representing the average calculated over a batch of time steps  $t$  during gameplay.
- $r_t(\theta)$ : The probability ratio (defined above).
- $\hat{A}_t$ : The estimated Advantage.
- $\text{clip}(.,.)$ : A mathematical function that restricts the probability ratio from exceeding a specific range.
- $\epsilon$ : The clipping parameter (typically set to 0.2). This acts as a safeguard, ensuring the policy never changes by more than 20% in a single step.

**The Total Objective Function.** During training, the agent does not merely improve its decision-making (the Actor); it simultaneously improves its board evaluations (the Critic) and forces itself to explore. These elements are combined into the total objective function:

$$L_t^{TOTAL}(\theta) = \hat{\mathbb{E}}_t \left[ L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t) \right],$$

where:

- $L_t^{TOTAL}(\theta)$ : The complete objective function that the PPO algorithm maximizes.
- $L_t^{CLIP}(\theta)$ : The actor's clipped objective, which safely improves the agent's strategy.
- $L_t^{VF}(\theta)$ : The Value Function loss. This measures the error in the Critic's predictions. By subtracting this loss, the algorithm trains the Critic to better evaluate board states.
- $c_1$ : A weighting coefficient determining how much focus the algorithm places on training the Critic.
- $S[\pi_\theta](s_t)$ : The Entropy bonus. This measures the randomness of the policy. It rewards the agent for keeping its probability distribution spread out, encouraging exploration instead of repeating the exact same sequence of moves.
- $c_2$ : A weighting coefficient determining the strength of the exploration incentive.

### 2.3.3 Maskable PPO

The action space of the Catan environment contains 290 discrete actions. However, at any given moment, only a subset of these actions is valid due to game rules. For example, the agent cannot build structures without the required resources or place roads in invalid positions.

To address this issue, the Maskable PPO algorithm was used. Maskable PPO is an extension of the standard PPO algorithm that supports action masking during policy evaluation.

At each decision step, the environment provides a binary mask  $m_t$  indicating which actions are legal ( $m_{t,i} = 1$ ) and which are illegal ( $m_{t,i} = 0$ ). Instead of penalizing the agent for choosing an invalid action, the mask is applied directly to the raw output (logits), denoted as  $z_i$ , of the policy network’s final layer *before* the Softmax activation function.

The masked logits  $z'_i$  are calculated as:

$$z'_i = \begin{cases} z_i & \text{if } m_{t,i} = 1 \\ -\infty & \text{if } m_{t,i} = 0 \end{cases}$$

When these masked logits are passed through the Softmax function, the exponential of negative infinity becomes exactly zero ( $e^{-\infty} = 0$ ). This ensures that invalid actions are assigned a strict 0% probability, and the policy distribution  $\pi_{\theta}(a_t|s_t)$  remains a valid probability distribution summing to 1 across only the legal actions.

This approach offers two main advantages. First, it reduces the effective action space that the agent must explore. Second, it eliminates the need for the agent to learn the game rules through trial-and-error penalties. Instead, the agent can focus entirely on learning strategic decision-making.

### 2.3.4 Expert-Guided Reinforcement Learning

Exploration in complex environments such as *Settlers of Catan* can be difficult for standard reinforcement learning algorithms due to the large state space and sparse reward signals. To accelerate learning, an expert-guided reinforcement learning mechanism was introduced.

During training, a heuristic expert policy evaluates all valid actions available to the agent. The expert is implemented using the ValueFunctionPlayer from the Catanatron framework, which performs a one-step lookahead by simulating each candidate action and evaluating the resulting state using a handcrafted value function.

Each action receives a numerical score representing its estimated quality according to the expert. These scores are normalized to the interval  $[0, 1]$ , and the reinforcement learning agent receives an additional reward bonus proportional to the normalized score of the chosen action. This provides a continuous signal encouraging the agent to prefer actions that are considered strong by the expert while still allowing independent exploration.

The expert bonus is scaled during training, starting at 0.2 and gradually decaying to a minimum value of 0.05. Importantly, the expert signal remains smaller than the terminal win/loss reward, ensuring that the agent ultimately optimizes for

winning the game rather than merely imitating the expert.

The total reward received by the agent at each step can therefore be expressed as the sum of the base environment reward and the expert bonus:

$$r_t^{\text{total}} = r_t^{\text{env}} + \lambda \cdot s_{\text{expert},t},$$

where  $r_t^{\text{env}}$  represents the reward provided by the game environment (as defined previously in Section 2.2.4),  $s_{\text{expert},t}$  is the normalized expert score for the selected action at step  $t$ , and  $\lambda$  is a scaling factor controlling the influence of the expert guidance during training.

### 2.3.5 Imitation Learning Initialization (Behavioral Cloning)

While the continuous reward shaping described in the previous section provides a directional signal, it remains inefficient for learning the base mechanics of a game as complex as *Settlers of Catan*. Under the expert-guided paradigm, the agent still begins with a randomly initialized policy and must rely on trial-and-error to discover valid sequences of actions. As documented in reinforcement learning literature, explicitly shaping rewards often introduces the risk of “reward hacking,” where the agent learns to exploit the intermediate bonuses rather than achieving the actual objective. The model wastes computational resources attempting to learn elementary strategies from scratch, such as the absolute necessity of placing initial settlements on high-yield intersections.

To circumvent this inefficiency, a two-stage training pipeline based on Imitation Learning was implemented. This approach draws directly from landmark successes in AI board game research, such as the AlphaGo system [SHM<sup>+</sup>16], which demonstrated that “warm-starting” a policy network through supervised learning on expert demonstrations drastically reduces the sample complexity required for subsequent reinforcement learning.

Specifically, Behavioral Cloning (BC) was utilized to initialize the actor network’s weights before the agent interacted with the environment via PPO. By framing the initial learning phase as a supervised learning problem, the agent directly inherits the foundational knowledge of the game without the need for blind exploration. The process consists of two primary stages:

1. **Data Collection.** A dataset of expert demonstrations, denoted as  $\mathcal{D} = \{(s_1, a_1), (s_2, a_2), \dots, (s_N, a_N)\}$ , is generated by having the heuristic *ValueFunctionPlayer* (whose relative performance is detailed in Table 2.1) play thousands of simulated games. At each decision step  $t$ , the environment state  $s_t$  and the discrete action chosen by the expert  $a_t$  are recorded.
2. **Supervised Pre-training.** Before any reinforcement learning takes place, the agent’s policy network (the Actor) is trained to mimic the expert’s decisions. The network is optimized to predict the expert action  $a$  given the state  $s$  by minimizing the negative log-likelihood, commonly formulated as the Cross-Entropy Loss:

$$L_{BC}(\theta) = -\mathbb{E}_{(s,a) \sim \mathcal{D}}[\log \pi_{\theta}(a|s)]$$

where  $\pi_\theta(a|s)$  represents the probability assigned by the policy network with parameters  $\theta$  to the expert’s chosen action. Action masking is strictly applied during this forward pass to ensure the network only learns from valid action distributions and ignores illegal moves.

Once the network converges to an acceptable level of accuracy in predicting the expert’s moves, this behavioral cloning phase concludes. The pre-trained weights are then transferred to the PPO algorithm. From this point forward, the agent learns purely from the environment’s Dense VP reward (as defined in Section 2.2.4) and no longer relies on the expert heuristic. This allows the reinforcement learning algorithm to start exploration from a structurally sound baseline, giving it the potential to fine-tune its strategy and eventually surpass the capabilities of the heuristic teacher.

## 2.4 Training Setup

Training was performed using the Catanatron environment integrated with the Stable Baselines3 reinforcement learning framework. The agent was trained using the Maskable PPO algorithm with a convolutional neural network designed to process the spatial representation of the game board.

To evaluate the efficacy of imitation learning and different reward structures, two distinct training pipelines were established:

- **Shaped Reward Pipeline:** The agent was trained entirely from scratch using standard Maskable PPO and guided by the Shaped Environment Reward (as defined in Section 2.2.4).
- **Imitation Learning Pipeline:** The agent’s policy network was first initialized using Behavioral Cloning on a dataset of expert matches (as detailed in Section 2.3.5). Following this supervised pre-training, the network weights were transferred to Maskable PPO, and the agent was fine-tuned using only the sparse Dense VP Reward.

Training was performed for multiple million environment steps, during which the agents continuously improved their policies through interaction with the environment and feedback from their respective reward functions. The experiments were executed on a high-performance computing cluster (Vega) equipped with GPU acceleration to handle the massive sample complexity required by the environment.

## 2.5 Baseline Opponent

The Catanatron framework provides several built-in agents with different levels of playing strength. Table 2.1 summarizes the approximate relative performance of these agents in head-to-head matches as reported in the official Catanatron documentation [Col23b].

Among these agents, the **ValueFunctionPlayer** was selected as the baseline opponent in this work. This agent evaluates candidate actions using a handcrafted

heuristic value function combined with a one-step lookahead simulation. It provides a reasonably strong opponent while remaining computationally efficient, unlike search-based agents such as AlphaBeta. This efficiency is important for large-scale reinforcement learning experiments that require millions of simulated games.

| Player               | Win Rate | Opponent             | Games |
|----------------------|----------|----------------------|-------|
| AlphaBeta (n=2)      | 80%      | ValueFunction        | 25    |
| <b>ValueFunction</b> | 90%      | GreedyPayouts (n=25) | 25    |
| GreedyPayouts (n=25) | 100%     | MCTS (n=100)         | 25    |
| MCTS (n=100)         | 60%      | WeightedRandom       | 15    |
| WeightedRandom       | 60%      | Random               | 1000  |
| WeightedRandom       | 50%      | VictoryPoint         | 1000  |
| VictoryPoint         | 60%      | Random               | 1000  |
| Random               | –        | –                    | –     |

Table 2.1: Relative strength of several built-in Catanatron agents in 1v1 games. Results reproduced from the Catanatron documentation [Col23b].

# Chapter 3

## Experimental Results and Discussion

As concluded in the methodological analysis of Chapter 2, traditional value-based Reinforcement Learning methods are unsuited for *Settlers of Catan*. The game’s astronomical branching factor and state-action space require severe, unrealistic simplifications for value-based algorithms to converge. Therefore, there is no further need to examine value-based methods in this work; instead, this research shifts focus entirely to policy-optimization algorithms.

To systematically answer the research questions posed in Chapter 2, three core training methodologies were evaluated. Within each of these methodologies, various reward functions and hyperparameters were tested to explore different learning scenarios and identify the most stable configurations. For clarity throughout this chapter, the core approaches are categorized under the following shortnames:

1. **Pure-PPO:** The baseline approach where the agent is trained entirely from scratch using standard Maskable PPO. Throughout the experiments, different reward formulations (such as the Dense VP reward) and hyperparameter configurations were tested to establish the agent’s baseline learning capabilities without any expert intervention.
2. **EG-PPO (Expert-Guided PPO):** An approach where the agent learns from scratch using Maskable PPO, but the training is augmented with expert guidance. This methodology evaluates how integrating heuristic feedback and shaped rewards impacts the learning curve across various parameter settings.
3. **BC-PPO (Behavioral Cloning PPO):** A two-stage approach where the policy network is first initialized via supervised Imitation Learning on an expert dataset. Following this pre-training phase, the agent is fine-tuned using Maskable PPO, experimenting with different reward functions during the reinforcement learning phase to optimize its final strategy.

The following sections dissect the performance of these approaches across the different tested scenarios. By examining the training metrics across millions of environment steps, we analyze the learning dynamics, sample efficiency, and stability of the agents during the training phase. We observe how different reward formulations, hyperparameter configurations, and initialization strategies influenced the convergence of the policy networks and the final in-game strategies discovered by the agents. Finally, Section 3.6 compares these results against related work on *Settlers of Catan* and provides definitive answers to the core research questions.

### 3.1 Pure-PPO Dynamics: Establishing the Baseline

In this initial phase, the Pure-PPO agent was trained entirely from scratch without any expert intervention. To rigorously establish the baseline capabilities of the algorithm, various configurations were tested across multiple independent axes: neural architecture, reward formulation, and training curriculum (such as self-play). All representative experimental runs in this phase were trained for 10 million environment steps against the heuristic *ValueFunctionPlayer* (VFP) on the standard *Settlers of Catan* board.

Rather than detailing every iterative run, Table 3.1 highlights the four most critical experiments that define the limitations of the Pure RL approach.

| Version | Experimental Axis Tested | Key Variation                      | Win Rate |
|---------|--------------------------|------------------------------------|----------|
| v2      | Baseline Reference       | CNN + 4-layer MLP, standard setup  | 8.2%     |
| v6      | Neural Architecture      | Graph Neural Network (GNN - GATv2) | 9.6%     |
| v7      | Reward Shaping           | Production-aware step rewards      | 5.2%     |
| v9      | Curriculum / Opponents   | Pure cyclic self-play from scratch | 3.0%     |

Table 3.1: Representative Pure-PPO runs at 10M steps, isolating different experimental axes.

**The Baseline Reference (v2).** Version 2 serves as the standard baseline, directly mirroring the foundational architecture introduced in Pull Request 297 of the core environment repository. Utilizing a Convolutional Neural Network (CNN) feature extractor paired with a mathematically right-sized 4-layer Multi-Layer Perceptron (MLP), this run achieved an 8.2% win rate against the VFP. By representing the established state-of-the-art for this specific repository, v2 serves as the definitive reference point against which all subsequent architectural and environmental modifications in this research were measured.

**Architectural Modifications (v6).** To test if the network structure was bottlenecking performance, version 6 replaced the standard CNN with a Graph Attention Network (GATv2). Because the Catan board is inherently a graph of hexes and intersections, a GNN is the most theoretically motivated architecture for this environment. However, the GNN achieved a 9.6% win rate, only a marginal improvement over the baseline CNN. This indicates that the regular hex grid of Catan is easily captured by standard convolutions, and neural architecture is not the primary limiting factor in the agent’s learning process.

**Reward Shaping Attempts (v7).** Version 7 attempted to manually shape the agent’s behavior by explicitly rewarding resource production, a metric heavily prioritized by the expert VFP. Counterintuitively, this richer, production-aware reward signal actually degraded performance, dropping the win rate to 5.2%. This suggests that providing the agent with richer observation and reward signals yields no benefit without a corresponding mechanism to guide exploration; the agent simply receives information it cannot effectively act upon.

**The Failure of Self-Play (v9).** Version 9 tested a pure cyclic self-play curriculum from scratch. While self-play can be a powerful mechanism for policy refinement, applying it to an agent with absolutely no prior knowledge of *Settlers of Catan* proved premature. Initiating self-play from a random initialization in such

a highly stochastic, sparse-reward environment produced the worst result of the entire project (a 3.0% win rate). Because the baseline agent lacked any foundational understanding of basic game mechanics, the self-play loop rapidly converged to degenerate strategies. Rather than learning objectively sound Catan principles, the agents simply learned to overfit and exploit each other’s random, untrained weaknesses.

**The Credit Assignment Ceiling.** Across all Pure-PPO variants tested, regardless of architecture, reward function, or opponent distribution, the win rate firmly hit a ceiling at approximately 10%. No single independent axis was able to move the needle beyond this threshold. This ceiling is a direct manifestation of the temporal credit assignment problem inherent to *Settlers of Catan*: with approximately 400 sequential decision points per game, sparse terminal win/loss conditions, and an action space peaking at 290 valid moves, the gradient signal reaching early game decisions is simply too weak to guide meaningful learning from a random initialization.

This insurmountable plateau proves that architecture and manual reward shaping address the wrong bottleneck, setting the stage for the introduction of Expert Guidance (EG-PPO).

## 3.2 Direct Comparison: Pure-PPO vs. EG-PPO

To isolate the impact of the expert-guided reward shaping, a controlled ablation study was conducted. Both the Pure-PPO and EG-PPO agents were trained using an identical hyperparameter configuration over 30 million environment steps. The sole difference between the runs was the inclusion of the expert guidance mechanism in the EG-PPO environment.

This ablation study used a sparse, terminal-only reward function rather than step-by-step dense feedback. The agents received a single scalar reward at the end of the game based on the win/loss outcome and the final Victory Point margin. Following the 30 million training steps, both models were evaluated in 500 head-to-head matches against a baseline heuristic opponent. The comparative results are detailed in Table 3.2.

| Agent Model         | Win Rate             | Avg VP      | Settlements | Cities | Roads | Army |
|---------------------|----------------------|-------------|-------------|--------|-------|------|
| Pure-PPO (Baseline) | 2.2% (11/500)        | 4.14        | 2.06        | 0.36   | 0.31  | 0.15 |
| EG-PPO (Guided)     | <b>5.6%</b> (28/500) | <b>4.56</b> | 1.74        | 1.11   | 0.30  | 0.00 |

Table 3.2: Ablation study results across 500 evaluation games using a sparse, terminal-only reward function.

**The Impact of Reward Sparsity.** The absolute win rates for both agents were low under this configuration. This is directly attributable to the sparse terminal reward, which creates a severe temporal credit assignment problem. Because the agent only receives feedback after hundreds of decision steps, it struggles to identify which early-game actions contributed to the final outcome. However, the relative performance clearly validates the expert guidance: EG-PPO successfully mitigated this sparsity, more than doubling the win rate of the baseline agent.

**Strategic Divergence and the Local Optimum Trap.** Beyond raw performance, the metrics reveal a fundamental divergence in learned strategies. The Pure-PPO baseline fell into a local optimum, relying on purchasing Development Cards (averaging 0.15 Army size) for easy early points while neglecting long-term infrastructure. Conversely, the EG-PPO agent broke out of this trap. Guided by the heuristic, it abandoned the Development Card strategy entirely in favor of heavy infrastructure investment, building over three times as many cities as the baseline (1.11 compared to 0.36).

### 3.3 Expert-Guided PPO: Breaking the Ceiling

Having established the baseline limitations of Pure RL and validated the stabilizing effect of heuristic guidance in the ablation study, the next phase of experiments fully integrated Expert Guidance (EG-PPO) alongside dense step-wise rewards. By providing the agent with continuous, heuristically weighted feedback, the fundamental credit assignment bottleneck was bypassed.

Table 3.3 details the progression of the EG-PPO agent across extended training horizons.

| Version (Steps) | Win Rate     | Avg VP      | Settles | Cities | Army | Dev VP |
|-----------------|--------------|-------------|---------|--------|------|--------|
| v10 (10M)       | 12.6%        | 5.37        | 1.49    | 0.97   | 0.37 | 0.87   |
| v13 (10M)       | 13.0%        | 6.15        | 1.17    | 1.04   | 0.65 | 1.54   |
| v15 (10M)       | 15.4%        | 6.16        | 1.12    | 1.06   | 0.63 | 1.60   |
| v19 (30M)       | 21.2%        | 7.03        | 1.00    | 1.17   | 0.80 | 2.07   |
| v25 (35M)       | 28.4%        | 7.19        | 1.35    | 1.27   | 0.70 | 1.60   |
| v26 (50M)       | <b>31.2%</b> | <b>7.23</b> | 1.49    | 1.27   | 0.64 | 1.39   |
| v28 (60M)       | 28.6%        | 6.73        | 2.13    | 1.14   | 0.28 | 0.78   |

Table 3.3: Progression of EG-PPO agents evaluated over 500 games against the heuristic baseline. The integration of expert guidance immediately breaks the 10% Pure-PPO ceiling and successfully scales with additional compute.

**Immediate Breakthrough and Scaling.** The most immediate observation from the EG-PPO runs is the shattering of the 10% performance ceiling. Even at only 10 million training steps (v10, v13, v15), the expert-guided agents consistently achieved win rates between 12.6% and 15.4%. More importantly, unlike the Pure-PPO approach which stagnated, the EG-PPO framework successfully scaled with increased compute. By extending the training horizon to 30 million (v19) and 50 million steps (v26), the win rate reached 31.2%, more than triple the best Pure-PPO baseline.

**Emergence of Strategic Diversity.** Beyond raw performance, the extended EG-PPO runs demonstrated that the agent was actively exploring and refining distinct, viable paths to victory rather than getting permanently trapped in a single local optimum. For example, version 26 (the highest performing agent at 31.2%) heavily favored a Development Card strategy, averaging 1.39 Dev VPs and an Army size of 0.64. Conversely, version 28 (which trained for 60 million steps and achieved a competitive 28.6% win rate) exhibited a radically different playstyle. It largely ignored Development Cards (0.78 Dev VP, 0.28 Army) in favor of aggressive board expansion, averaging 2.13 settlements per game. This strategic divergence proves that the

expert guidance did not force the agent into a rigid, scripted playstyle; rather, it provided the stable foundation necessary for the PPO algorithm to discover its own high-level strategies.

**The Need for Behavioral Cloning.** While expert guidance successfully steered the agent away from flawed short-term traps and pushed the win rate above 30%, relying on Reinforcement Learning from scratch remains computationally inefficient. Even with shaped rewards and heuristic hints, the agent still requires upwards of 50 million steps (as seen in v26) to discover basic *Settlers of Catan* fundamentals, such as optimal initial settlement placement.

This limitation necessitates the third approach investigated in this research: Behavioral Cloning (BC-PPO). By initializing the network with expert human data, the model can entirely bypass this foundational learning phase, entering the PPO training loop with an already-developed understanding of initial placements and economic scaling.

### 3.4 Behavioral Cloning and Fine-Tuning (BC-PPO)

While the Expert-Guided (EG-PPO) approach successfully broke the baseline limitations, it ultimately plateaued at a 31.2% win rate (v26). Increasing compute, altering reward functions, and introducing self-play leagues failed to breach this ceiling. Analysis of this plateau revealed a structural limitation: the expert guidance mechanism explicitly modifies the reward function to incentivize actions preferred by the baseline heuristic. Consequently, the agent is constantly biased toward the heuristic’s 1-ply strategy throughout the entire training run, effectively penalizing the agent for attempting to discover superior, non-heuristic actions.

To decouple the injection of expert knowledge from the reinforcement learning objective, a two-phase pipeline was implemented: an offline Behavioral Cloning (BC) phase to initialize the network with expert fundamentals, followed by unconstrained PPO fine-tuning using only environment rewards.

#### Phase 1: Behavioral Cloning (Pre-training)

Expert trajectories were collected by running the *ValueFunctionPlayer* (VFP) against itself for 100,000 games, generating approximately 15 million labeled decision points. The policy network, sharing the exact CNN and MLP architecture as the PPO agents, was trained using supervised learning to minimize the cross-entropy loss between its action logits and the expert’s chosen actions.

#### Phase 2: Unconstrained PPO Fine-Tuning

Following BC initialization, the policy was loaded into the PPO framework. The expert guidance wrapper was completely removed. The agent was optimized over 80 to 170 million steps using only the *dense\_vp\_rewards* signal, granting it total freedom to deviate from the expert’s strategy whenever doing so increased the expected return.

The results of this decoupled pipeline are detailed in Table 3.4.

#### Shattering the Performance Ceiling

The effect of decoupling expert knowledge from the RL objective was immediate. Version 35 (80M PPO steps) achieved a 66.4% win rate against the baseline, more

| Version      | PPO Steps           | Win Rate     | VP   | Settles | Cities | Army | Dev VP |
|--------------|---------------------|--------------|------|---------|--------|------|--------|
| v26 (EG-PPO) | 50M (Ceiling)       | 31.2%        | 7.23 | 1.49    | 1.27   | 0.64 | 1.39   |
| v35 (BC-PPO) | 80M                 | <b>66.4%</b> | 8.95 | 2.14    | 1.89   | 0.55 | 1.09   |
| v36 (BC-PPO) | 120M (Warm Restart) | <b>68.8%</b> | 8.99 | 2.15    | 1.91   | 0.57 | 1.03   |
| v37 (BC-PPO) | 170M (Mixed Opps)   | <b>69.6%</b> | 9.08 | 2.07    | 1.95   | 0.57 | 1.08   |

Table 3.4: Progression of BC-PPO agents compared to the EG-PPO ceiling. The BC-PPO agents were pre-trained on 100,000 expert games before unconstrained fine-tuning.

than doubling the highest result achieved by the reward-shaping approach. Extended training through learning rate warm restarts (v36) and opponent diversification (v37) pushed this win rate even higher, approaching 70%.

### Strategic Diversification

The evaluation metrics in Table 3.4 highlight why the BC-PPO agents performed so much better than their EG-PPO counterparts. The reward-shaping agents overspecialized in Development Cards and Army size, constrained by the heuristic’s valuation. By contrast, the BC-PPO agents distributed their scoring far more evenly. They increased core infrastructure (averaging 2.1 settlements and 1.9 cities per game) while maintaining competitive secondary VP sources. By embedding expert knowledge in the initial parameters rather than the reward function, the agent was able to use the baseline heuristic as a starting floor rather than an absolute ceiling, resulting in a more robust and less exploitable overall strategy.

## 3.5 The Impact of Teacher Quality on Imitation Learning

While the BC-PPO pipeline utilizing the VFP heuristic successfully pushed win rates near 70%, the quality of the offline dataset remains bounded by the capabilities of the teacher. The VFP relies on a 1-ply lookahead with static heuristic weights, meaning its strategic limitations, such as an inability to model opponent behavior or plan multi-turn combinations, are inherently transferred to the agent during Phase 1.

To determine how the ceiling of the final PPO agent scales with the skill of the demonstrating expert, further experiments were conducted replacing the 1-ply VFP teacher with a multi-ply Alpha-Beta search agent. A natural hypothesis is that pre-training the agent on a strategically superior teacher would establish a higher performance floor and ultimately lead to an even stronger fine-tuned agent. As a point of reference, Table 2.1 illustrates that the AlphaBeta agent dominates the VFP in direct head-to-head matchups with an 80% win rate, although this strategic superiority comes at the cost of being four to six times slower due to the computational overhead of its multi-ply search tree.

To test this hypothesis, a new dataset of 100,000 games played entirely by the AlphaBeta agent (configured to a search depth of 2) was generated. Two new models were trained via Behavioral Cloning on this dataset before proceeding to unconstrained PPO fine-tuning: version 38 (PPO fine-tuned against the VFP) and version 39 (PPO fine-tuned against AlphaBeta). The comparative performance of these

models is detailed in Table 3.5.

| Model | BC Teacher             | PPO Opponent | PPO Steps | vs VFP | vs AB |
|-------|------------------------|--------------|-----------|--------|-------|
| v35   | VFP (100K games)       | VFP          | 80M       | 66.4%  | 65.4% |
| v38   | AlphaBeta (100K games) | VFP          | 80M       | 61.6%  | 60.0% |
| v39   | AlphaBeta (100K games) | AlphaBeta    | 50M       | 57.8%  | 56.4% |

Table 3.5: Win rates of BC-pretrained agents evaluated against VFP and AlphaBeta (depth=2). All evaluations consist of 500 head-to-head games.

### 3.5.1 The Paradox of the Superior Teacher

Counterintuitively, the evaluation results demonstrate an inverse relationship between the teacher’s search depth and the agent’s final performance. The model pre-trained on the simpler VFP teacher (v35) outperformed the models trained on the superior AlphaBeta teacher (v38 and v39). Version 38 experienced a roughly 5-percentage-point drop in win rate against both evaluation opponents, and version 39 dropped even further, dipping below 60%.

This performance degradation can be attributed to the fundamental mechanics of Behavioral Cloning and the problem of *unobserved intent*. The policy network operates as a 1-ply reactive function approximator: it maps the current, static observation state  $s_t$  to an immediate action probability distribution  $\pi(a_t|s_t)$ . The VFP operates under the exact same 1-ply constraint, making its decision-making logic transparent and easily replicable by the neural network.

Conversely, the AlphaBeta agent selects actions based on multi-step future simulations and anticipated opponent responses. Because this future search tree is completely invisible to the neural network during Behavioral Cloning, the AlphaBeta agent’s decisions often appear disjointed or “noisy” relative to the immediate state observation. The neural network attempts to map static board states to complex, multi-turn plans without the necessary contextual data, resulting in severe policy confusion. Consequently, the agent learns a flawed baseline policy that subsequent PPO fine-tuning struggles to correct.

### 3.5.2 Strategic Divergence

Table 3.6 illustrates exactly how this policy confusion manifested in the agents’ learned behaviors during actual gameplay.

The VFP-trained agent (v35) learned a consistent, scalable strategy centered on spatial expansion, averaging 2.14 settlements and 1.89 cities per game. Because the VFP’s greedy approach to resource acquisition is easy to imitate, v35 quickly mastered the game’s core economic engine.

By contrast, the AlphaBeta-trained agents (v38 and v39) exhibited stunted spatial expansion. Version 39, despite training against the stronger opponent, averaged only 1.64 settlements per game, well below v35. Unable to successfully replicate AlphaBeta’s complex multi-turn placement traps, the network instead defaulted to an over-reliance on secondary win conditions. Both v38 and v39 leaned much heav-

| Model | Eval Opp. | Avg VP | Settles | Cities | Road | Army | Dev VP |
|-------|-----------|--------|---------|--------|------|------|--------|
| v35   | VFP       | 8.95   | 2.14    | 1.89   | 0.43 | 0.55 | 1.09   |
| v35   | AB        | 8.81   | 2.23    | 1.79   | 0.34 | 0.58 | 1.14   |
| v38   | VFP       | 8.73   | 1.90    | 1.70   | 0.44 | 0.63 | 1.31   |
| v38   | AB        | 8.53   | 1.73    | 1.80   | 0.35 | 0.62 | 1.26   |
| v39   | VFP       | 8.52   | 1.64    | 1.86   | 0.37 | 0.57 | 1.27   |
| v39   | AB        | 8.45   | 1.75    | 1.83   | 0.30 | 0.61 | 1.22   |

Table 3.6: Play style statistics for BC-pretrained agents across evaluation opponents. Road = longest road indicator, Army = largest army indicator, Dev VP = average VP from development cards.

ier into purchasing Development Cards (averaging over 1.22 Dev VPs) and fighting for the Largest Army (averaging 0.60 Army size).

Ultimately, these experiments prove that in pure Behavioral Cloning pipelines without recurrent memory or search-augmented observations, teacher compatibility is more critical than raw teacher strength. A transparent, easily modeled heuristic provides a much stronger foundation for Reinforcement Learning fine-tuning than a highly complex search algorithm whose intent cannot be observed.

## 3.6 Discussion

The experiments detailed in this chapter produced a competitive Reinforcement Learning agent. To put these findings into perspective, this section benchmarks the final BC-PPO model against existing literature and directly answers the core research questions outlined in Chapter 2.

### 3.6.1 Comparison to Related Work

The results achieved by the final BC-PPO agent, peaking at a 69.6% win rate, represent a strong result relative to prior Reinforcement Learning work on *Settlers of Catan*. Historically, researchers have gravitated toward value-based methods, attempting to map state-action pairs to expected returns using variations of Q-learning. The outcomes of this thesis align with the performance ceilings observed in these prior attempts, confirming that policy optimization combined with expert pre-training is a more effective framework for this domain.

For instance, Whelan et al. [WMB23] investigated the application of Deep Q-Networks (DQN), Double DQN (DDQN), and Dueling Networks to the Catan environment. Similar to the Pure-PPO baseline experiments conducted in Section 3.1, they experimented with various model architectures and reward policies to handle the large action space and sparse feedback. Despite these variations, their value-based models struggled to grasp the game’s long-term strategic depth, achieving a win rate of only 3% against an Alpha-Beta baseline. This corroborates the structural limitations discussed in Chapter 2: value-based methods falter in Catan because the value of a specific action is highly contextual, leading to unstable function approximation.

Beyond early neural networks, the competitive Catan AI landscape has evolved to incorporate advanced search-based engines and hybrid reinforcement learning models. Research by Szita et al. [SCS10] demonstrated the efficacy of Monte Carlo Tree Search (MCTS) for navigating the game’s non-deterministic dice rolls and hidden information. Extending this line of work, concurrent open-source projects such as the *Canopy* engine [Cul24] are actively adapting state-of-the-art architectures, drawing heavily from AlphaZero and KataGo, which combine deep neural network evaluations with advanced MCTS to push computational performance boundaries.

The existence of these heavily optimized, search-augmented engines reinforces a core finding of this thesis: the ultimate performance ceiling of an Imitation Learning agent is directly linked to the quality of its teacher. The success of our BC-PPO agent, which approached a 70% win rate when fine-tuned from a simple 1-ply baseline, suggests a clear path for future research. By utilizing advanced algorithms, such as those implemented in *Canopy*, as the expert demonstrators during the Behavioral Cloning phase, future Reinforcement Learning agents could achieve greater strategic depth.

### 3.6.2 Answers to Research Questions

Based on the empirical data and theoretical analysis presented in this chapter, we can now provide definitive answers to the research questions established in Chapter 2:

Regarding **RQ1**, training an agent from scratch (Pure-PPO) utilizing only environmental win/loss signals proved practically infeasible for *Settlers of Catan*. The vast action space and extended episode lengths created a severe credit assignment problem, preventing the policy network from associating early-game strategic placements with late-game victories. Integrating heuristic expert guidance (EG-PPO) successfully bypassed this bottleneck by transforming the sparse reward landscape into a dense one. By continuously evaluating the agent’s actions against an expert’s baseline during online training, EG-PPO provided immediate, step-by-step reinforcement. This allowed the agent to rapidly learn fundamental game mechanics (such as the necessity of building roads and settlements) and achieve baseline competence, proving that dense reward shaping is a prerequisite for online policy optimization in environments of this complexity.

In what concerns **RQ2**, we observed that while EG-PPO successfully stabilized training, its sample efficiency was constrained by an initial random exploration phase. Initializing the policy network via Behavioral Cloning (BC-PPO) improved both sample efficiency and the absolute performance ceiling. By absorbing the equivalent of 100,000 expert games offline, the BC-PPO agent began its online PPO fine-tuning phase with an already competent policy. This completely eliminated the need for random exploration, allowing the PPO algorithm to immediately focus on optimizing and refining an already functional strategy. Ultimately, this Imitation Learning pipeline yielded the highest recorded performance in this research, peaking at a 69.6% win rate against the heuristic baseline.

Counterintuitively, in response to **RQ3**, this research demonstrated that in the context of pure Behavioral Cloning without recurrent memory, training on a strategically superior multi-ply teacher actually degrades the final agent’s performance. While the Alpha-Beta agent easily defeats the 1-ply Value Function Player (VFP)

in head-to-head simulations, it proved to be a worse teacher. Because the neural network operates as a 1-ply function approximator, it suffered from *unobserved intent* when trying to imitate the Alpha-Beta agent; it could not see the hidden search tree motivating the expert’s decisions. By contrast, the 1-ply VFP’s reactive logic was completely transparent and easily modeled by the network. Therefore, the performance ceiling of a fine-tuned RL agent relies less on the absolute strategic strength of the demonstrator, and more on the architectural compatibility between the teacher’s decision-making process and the student’s observational capacity.

### 3.7 Future Research Directions

First, to resolve the issue of “unobserved intent” when learning from multi-ply search algorithms, future iterations of the policy network should integrate attention-based sequence models, such as Transformers [VSP<sup>+</sup>17]. By providing the agent with a temporal memory of past states and actions, the network could theoretically deduce the hidden, multi-turn strategies of advanced teachers like Alpha-Beta, allowing for successful Behavioral Cloning from much stronger datasets.

Second, with a strong foundational policy now established via Imitation Learning, future work should revisit self-play reinforcement learning. While pure cyclic self-play failed when initialized from scratch, applying a self-play curriculum to the BC-PPO agent could allow it to continuously refine its strategy and break past the limitations of its static heuristic teacher.

Finally, the reinforcement learning environment should be expanded back to its original 3-4 player format. Transitioning from a 1v1 configuration to a multi-agent setting introduces the complex dynamics of inter-player trading and negotiation, as well as increased unpredictability, since a full round involves many more intermediate decisions by opponents. Future research could explore multi-agent reinforcement learning (MARL) techniques to train agents capable of evaluating not just spatial placement, but the semantic value of proposing and accepting trades with human and artificial opponents.

## Chapter 4

# System Architecture and Application Deployment

While preceding chapters focused on the algorithmic development of Reinforcement Learning agents for *Settlers of Catan*, an artificial intelligence model is ultimately only as valuable as the system engineered to serve it. This chapter details the software engineering process of transitioning these trained policy networks from offline research artifacts into a fully deployed, interactive web application.

The primary objective is to transform an existing stateless simulation sandbox into a robust, real-time evaluation platform. This requires three core architectural upgrades: dynamic Stable-Baselines3 model integration without backend rebuilds, real-time event streaming for fluid gameplay, and a relational persistence layer for user authentication and match history.

### **Framework Disclosure and Original Contributions:**

To maintain focus on AI integration rather than recreating standard board game mechanics, this phase builds upon the open-source *Catanatron* framework [Col23a], which provides the domain game logic and basic graphical rendering. The original engineering contributions of this thesis consist of heavily refactoring and expanding this baseline by:

1. Replacing synchronous HTTP polling with a bidirectional WebSocket pipeline for real-time AI inference streaming.
2. Engineering a dynamic Stable-Baselines3 model registry to load ‘.zip’ checkpoints directly into the Application tier.
3. Implementing a PostgreSQL database and JSON Web Token (JWT) authentication flow for user persistence.
4. Integrating high-fidelity, AI-generated SVG biome assets into the React presentation tier to replace abstract graphics.

The chapter is structured as follows: Section 4.1 analyzes the baseline design, Section 4.2 details the implementation phase, Section 4.3 covers integration testing, Section 4.4 provides a practical user manual, and Section 4.5 summarizes outcomes and future enhancements.

## 4.1 Analysis of the Baseline System Architecture

Before discussing the engineered enhancements, it is necessary to analyze the foundational architecture of the upstream *Catanatron* framework to understand its structural limitations. From a software engineering perspective, the baseline system is structured as a monorepo adhering to a classic four-tier logical layout. The entire stack is containerized and orchestrated via Docker Compose [Doc26], ensuring consistent behavior across isolated environments.

Figure 4.1 illustrates the structural flow of data and execution across these four logical layers.

To strictly separate the game rules from the web infrastructure, the system is divided into the following interacting tiers:

- **Presentation Tier (React SPA):** Operating on the client-side, this tier is built with React and Material-UI. Its sole responsibility is to render the hexagonal board, capture human intents (such as clicking a node to build a settlement), and hold ephemeral user interface state. It strictly trusts the backend for game logic and communicates entirely via JSON payloads over HTTP.
- **Application Tier (Flask Backend):** Acting as the mediator, the stateless Flask REST API translates HTTP requests into in-process domain calls. Because the backend is stateless to allow for horizontal scaling, it does not hold the game in active memory. Instead, upon receiving a request, the Application tier queries the database, reconstructs the game state, executes the requested action, saves the new state, and returns the response to the client.
- **Domain Tier (Catanatron Core):** This is an isolated, pure Python library completely devoid of I/O operations. It encapsulates the mathematical formalization of the *Settlers of Catan* ruleset, defining legal actions, map generation, and the logic for all artificial intelligence opponents.
- **Data Tier (PostgreSQL & Filesystem):** For persistence, the system utilizes a PostgreSQL database operating as an append-only event log. Every discrete tick of the game (an action taken by a human or bot) is appended as a new row containing both human-readable JSON and a serialized binary payload. This binary blob allows the Application tier to rapidly reload the game state. Additionally, static assets, such as the pre-trained weights for the Reinforcement Learning agents, reside on the local filesystem.

While this baseline architecture is robust for isolated testing, its mechanism for advancing the game state was inherently inefficient for real-time human-AI interaction. In the baseline design, one synchronous HTTP request equates to exactly one player decision. Consequently, the backend possesses no background workers or push channels. When it is a bot's turn to play, the server does not autonomously calculate the move. Instead, the architecture relied on a *polling* mechanism: the React frontend was forced to repeatedly send empty POST requests to the server, prompting the AI to calculate and return its next move. If multiple bots were in the game, the client had to continuously poll the server until it was the human player's turn again. This strict request-response paradigm introduced significant latency, wasted network bandwidth, and resulted in a suboptimal user experience.

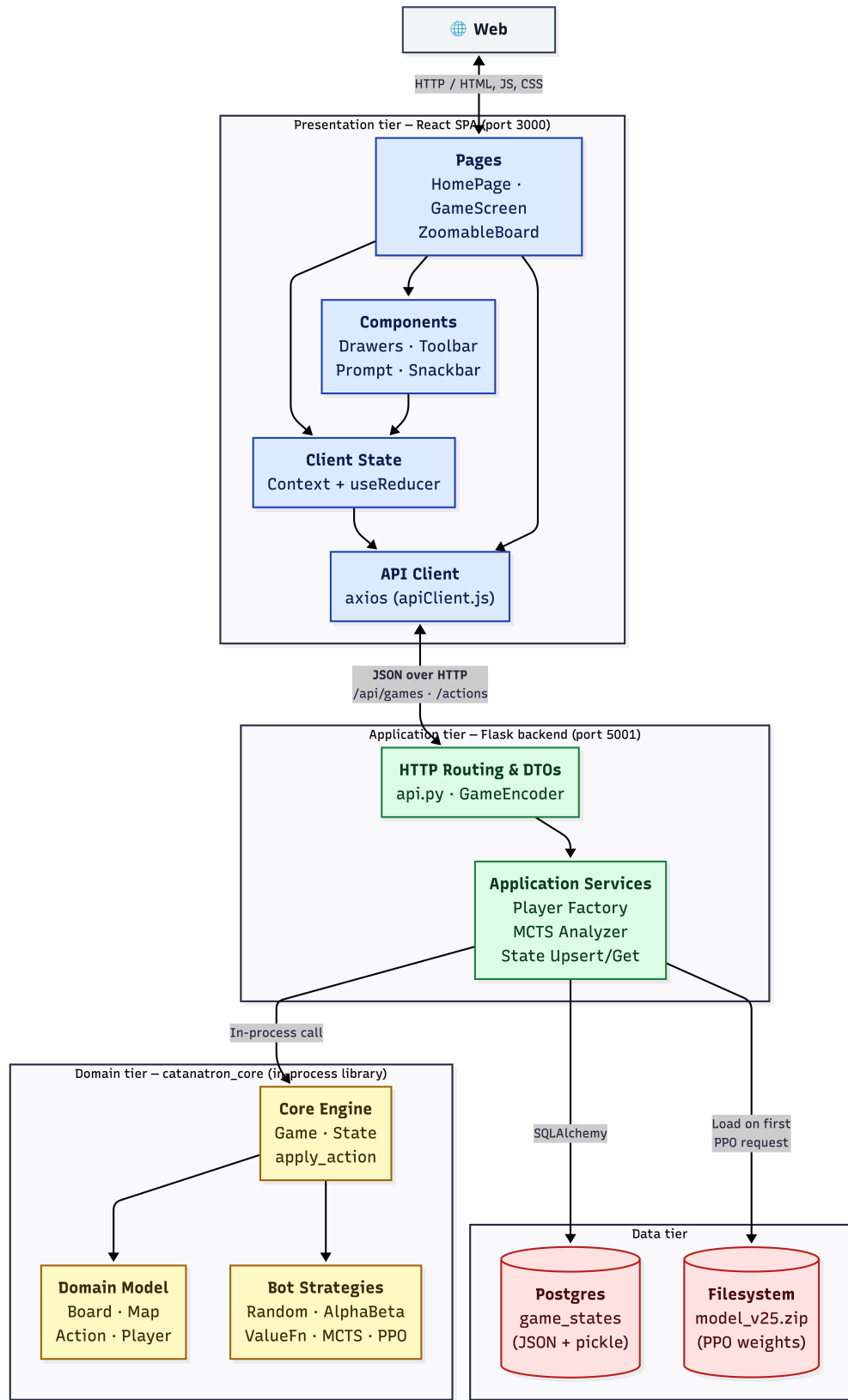


Figure 4.1: The baseline four-tier logical architecture of the web application.

that failed to capture the near-instantaneous inference capabilities of the trained BC-PPO agents.

The baseline simulator also lacked a persistent user identity system. Games were created anonymously, meaning the session identifier existed only temporarily in the frontend interface. If a player refreshed the page or lost connection, their link to the active game was permanently severed. Without a way to link games to specific accounts, it was impossible for users to resume interrupted matches or track their historical win rates against the AI models.

## 4.2 Implementation

Translating a theoretical machine learning model into a robust, production-ready application requires rigorous software engineering. This section details the practical implementation phase, highlighting the specific technologies, libraries, and code structures engineered to upgrade the baseline system.

In the initial framework [Col23a], the integration of Reinforcement Learning models was statically defined, requiring the source code to be modified and the container rebuilt to evaluate different algorithmic iterations. To resolve this limitation, a dynamic model registry was engineered into the Python Application tier using Flask. The REST API exposes a GET /api/players endpoint, driving the in-browser opponent selection interface. When a game session is initialized, the backend dynamically loads the corresponding Stable-Baselines3 MaskablePPO checkpoint [RHG<sup>+</sup>21] from the filesystem. To avoid the cost of repeatedly reading large neural-network weight files from disk, the loader is wrapped in an LRU-bounded cache, so that the second game using the same model reuses the resident weights instantly.

Listing 4.1 demonstrates the core logic implemented to load these Stable-Baselines3 checkpoints directly into the gameplay loop.

Listing 4.1: Dynamic model loading with an LRU-bounded weight cache.

```
import os, threading
from functools import lru_cache

from catanatron_server.loader import _load_sb3_model  # MaskablePPO
               .load wrapper

class ModelRegistry:
    def __init__(self, entries):
        self._by_id = {e.id: e for e in entries}
        # LRU-bounded SB3 loader so memory cannot grow unbounded
        # across many distinct PPO checkpoints.
        max_loaded = int(os.environ.get("MAX_LOADED_MODELS", "2"))
        self._load = lru_cache(maxsize=max_loaded)(_load_sb3_model)
        self._lock = threading.Lock()  # SB3.load is not thread-
            safe

    def create_player(self, player_id, color):
        desc = self._by_id[player_id]
        if desc.kind == "ppo":
```

```

        with self._lock:
            model = self._load(desc.model_path)
            return PPOPlayer(color, model=model)
        return _build_heuristic(desc, color)

```

Alongside dynamic model loading, the artificial latency introduced by HTTP polling had to be eliminated. To achieve this, the event-driven architecture was fundamentally restructured to utilize persistent, bidirectional WebSocket connections via the Flask-SocketIO library. When a human player submits an action via the existing `POST /api/games/<id>/actions` endpoint and the game state transitions to an artificial agent's turn, the server spawns a background task that drives the bot loop and pushes each resulting state update to all clients subscribed to the game room. The originating HTTP request returns immediately, so the human's browser is never blocked waiting for inference.

Listing 4.2 illustrates the backend implementation of this event streaming. The `start_bot_turns_if_needed` helper hands the bot loop to `socketio.start_background_task`, which schedules it on a separate worker; the loop itself (`run_bot_turns`) then advances the game one tick at a time and emits each new state to the WebSocket room.

Listing 4.2: Server-driven bot loop pushing each tick to subscribed clients.

```

from flask import current_app
from flask_socketio import SocketIO, join_room

socketio = SocketIO(cors_allowed_origins="*", async_mode="threading")

def emit_state(game):
    socketio.emit("state_updated", encode(game), to=f"game:{game.id}")

def start_bot_turns_if_needed(app, game_id):
    """Spawn run_bot_turns on a background worker so the caller
    returns at once."""
    socketio.start_background_task(run_bot_turns, app, game_id)

@socketio.on("subscribe")
def handle_subscribe(data):
    """Client joins a game room; any pending AI turns are resumed."""
    join_room(f"game:{data['game_id']}")
    emit_state(get_game_state(data["game_id"]))
    start_bot_turns_if_needed(current_app._get_current_object(),
                              data["game_id"])

def run_bot_turns(app, game_id):
    """Background task: advance bot turns, pushing each over
    WebSocket."""
    with app.app_context():
        while True:
            game = get_game_state(game_id)
            if game.winning_color() or not game.state.

```

```
current_player().is_bot:
    break
game.play_tick()
upsert_game_state(game)
emit_state(game)
socketio.sleep(0.2)
```

Beyond real-time gameplay, elevating the application into a data-driven platform required a relational user identity model. The PostgreSQL database schema was expanded to include a secure users table, linked via foreign-key constraints to a new game\_ownerships table that maps authenticated users to their persistent game sessions (Figure 4.2). Authentication is securely managed via a stateless JSON Web Token (JWT) flow [JBS15].

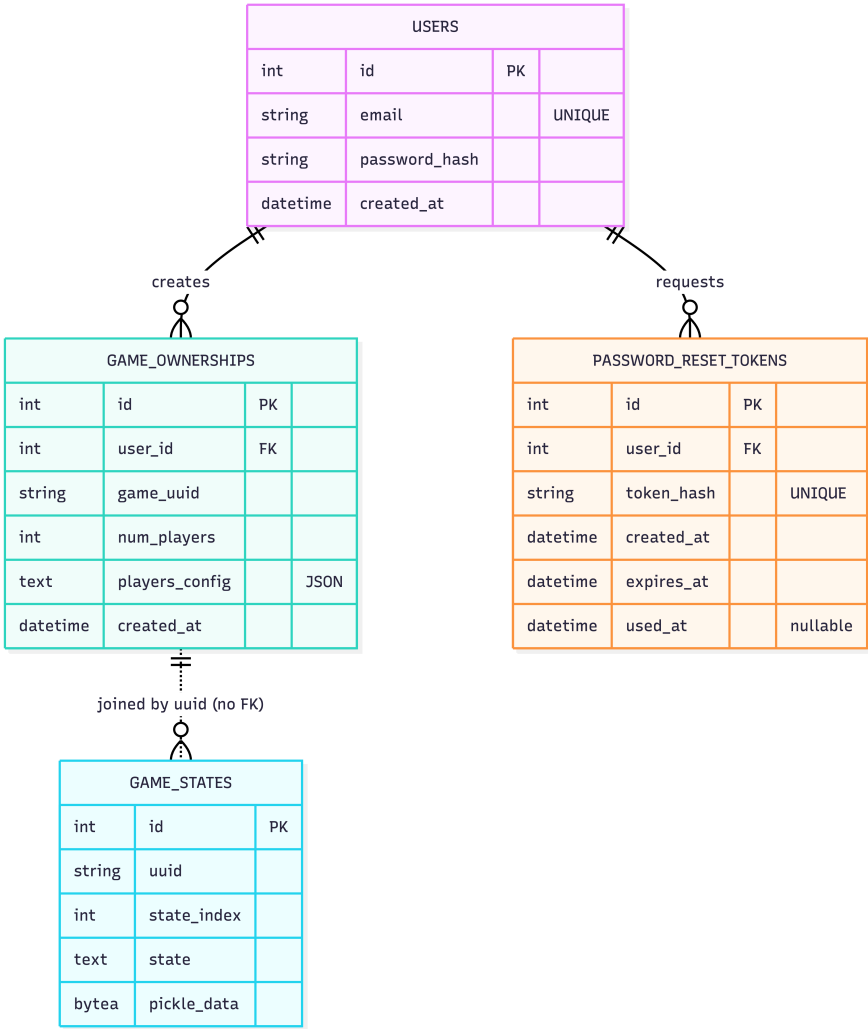


Figure 4.2: The relational database schema demonstrating the mapping between authenticated users and persistent game sessions.

Finally, the React presentation tier was heavily refined to increase user immersion. The baseline’s abstract, circuit-themed hexagonal tiles were replaced with high-fidelity, artificial intelligence-generated SVG (Scalable Vector Graphics) tiles representing classic geographical biomes (Figure 4.3). Because these assets are purely

vector-based, they integrate directly into React’s Virtual DOM, allowing the board to scale flawlessly across different resolutions.

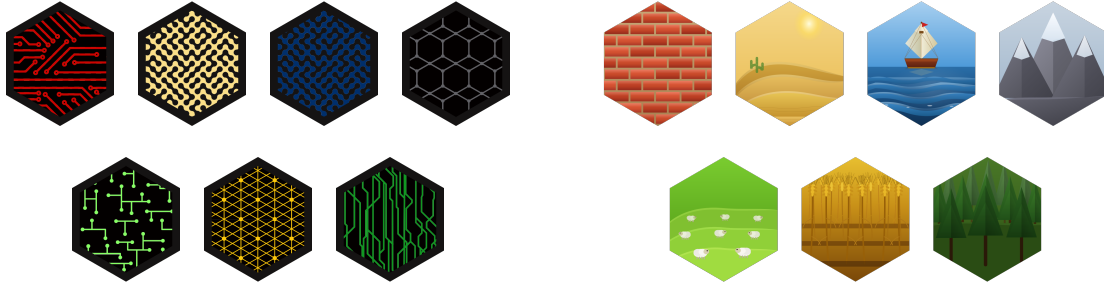


Figure 4.3: Comparison of the presentation tier assets. Left: The original circuit-board tiles (Reproduced from [Col23a]). Right: The newly integrated, AI-generated SVG biome tiles.

## 4.3 Testing

Testing is a fundamental activity in the software development lifecycle, ensuring system reliability, security, and adherence to functional requirements. Because the foundational architecture was inherited, the pre-existing core functionalities of the project were already rigorously verified by unit tests provided in the upstream repository. Therefore, the testing strategy for this engineering phase focused strictly on validating the newly implemented architectural features.

To achieve this, a focused suite of integration tests was developed in the `test-s/integration_tests/` directory. Using the Pytest framework, each test instantiates the full Flask application via the `create_app` factory configured against an isolated, in-memory SQLite database (`{"TESTING": True}`). This ensures that test runs are reproducible and never touch or corrupt the production database. The suite exercises the extended application through Flask’s `test_client` for HTTP routes and `socketio.test_client` for WebSocket flows, successfully validating the correctness of both the newly engineered RESTful API and the real-time event streaming layer.

## 4.4 User Manual

To provide a practical overview of the developed platform, this section details the primary user flows and interfaces of the web application.

### Authentication and Session Management

As detailed in Section 4.2, the application now supports persistent user identities. After signing in, users land on a personal dashboard (Figure 4.4) listing all past and active game sessions linked to their account, ensuring that interrupted matches can be resumed across different devices.

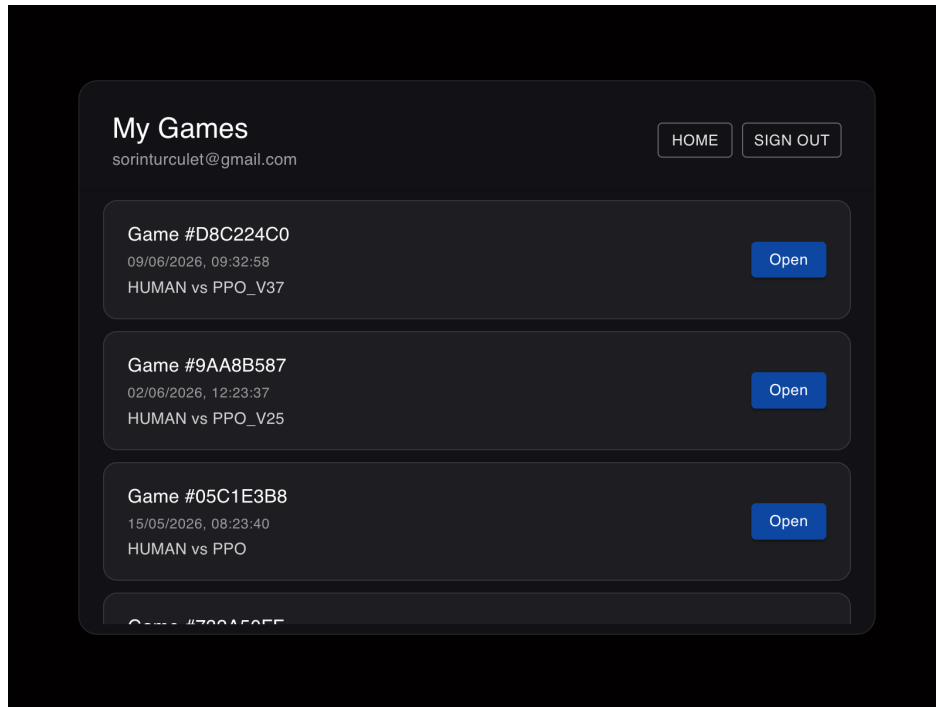


Figure 4.4: The user’s game history dashboard, listing prior sessions linked to the authenticated account.

### Game Initialization and Opponent Selection

Once authenticated, the user can initialize a new game. Leveraging the dynamic model registry engineered in the Application tier, the interface presents a selection menu of available artificial intelligence opponents (Figure 4.5). Users can choose between various heuristic baseline bots (e.g., *ValueFunctionPlayer*, *AlphaBeta*) and the newly trained Reinforcement Learning agents (e.g., BC-PPO) without requiring a backend server restart.

### Gameplay Interface

During active gameplay, the user interacts with the enhanced presentation tier (Figure 4.6). The board utilizes the newly generated high-fidelity SVG biome tiles, providing a clear, scalable, and visually appealing representation of the game state. The WebSocket integration ensures that actions taken by the human player are instantly processed, and the AI opponent’s inferences are pushed to the client and rendered in real-time without requiring manual page polling.

## 4.5 Conclusions and Future Enhancements

The primary objective of this engineering phase was to transition the trained BC-PPO agents from offline research artifacts into a fully deployed, interactive web application. By strategically analyzing the inherited *Catanatron* baseline, critical bottlenecks, such as synchronous HTTP polling and static model initialization, were identified and resolved.

The implementation of a persistent WebSocket architecture, a relational user identity model, and a dynamic Stable-Baselines3 model registry successfully trans-

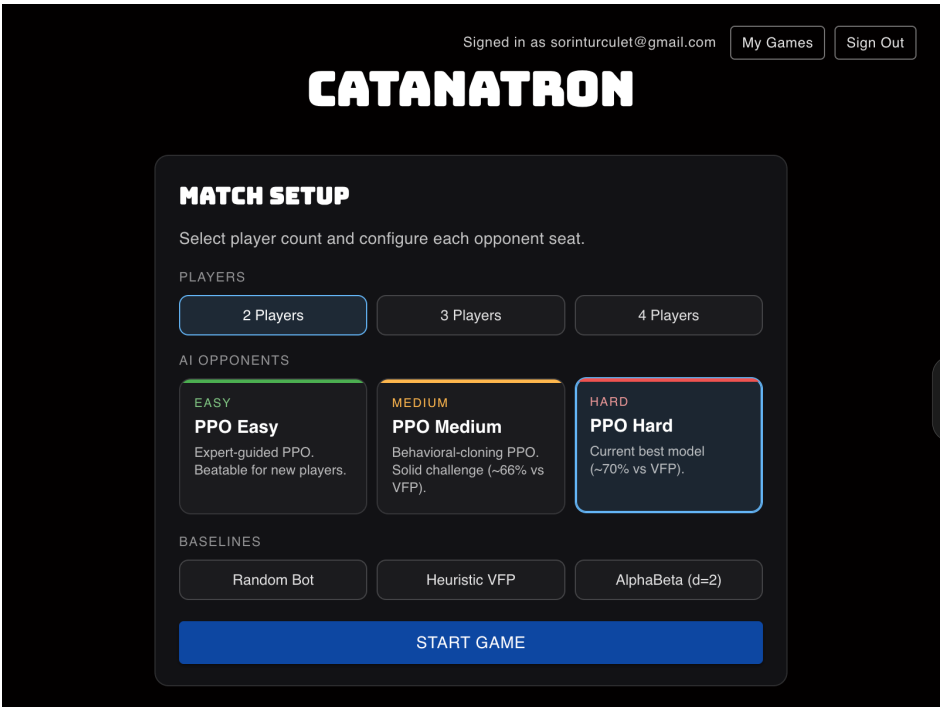


Figure 4.5: Game setup screen demonstrating the dynamic AI opponent selection.

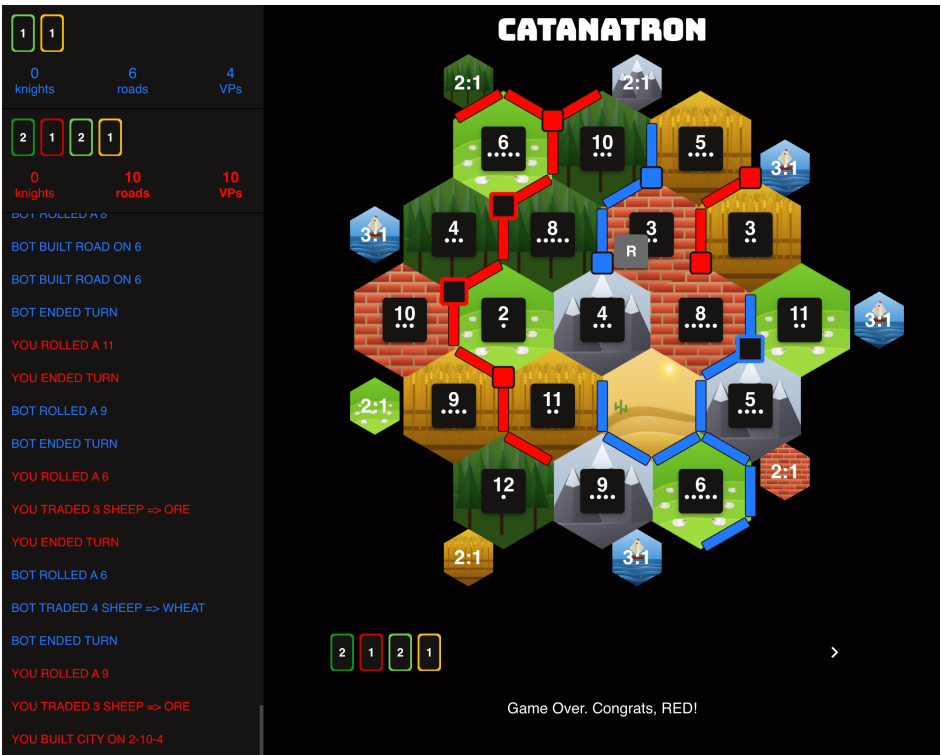


Figure 4.6: The main gameplay interface featuring the updated SVG board and real-time interaction.

formed the system into a robust evaluation platform. Rigorous integration testing validated these newly engineered components, ensuring the stability and correctness of the real-time gameplay loop. Ultimately, these architectural enhancements fulfill the established system requirements, providing a responsive interface where human gameplay can be directly evaluated against Imitation Learning models.

While the current architecture successfully resolves the baseline limitations, the newly established foundation opens several avenues for future development.

First, the architecture is now naturally positioned to support comprehensive multiplayer integration. The baseline simulator was strictly limited to single-player (human-versus-bots) gameplay due to its rigid HTTP polling constraints. However, because the bidirectional WebSocket pipeline removes this architectural blocker, future iterations of the platform could easily introduce room-based session management. This would enable mixed, real-time sessions where multiple human players can compete against each other alongside deployed artificial agents simultaneously.

Second, the integration of an Elo rating system [Elo78] represents a highly practical next step. Currently, this thesis evaluates the trained AI models by measuring their win rates against specific baseline bots. Implementing an Elo system would provide a more accurate way to benchmark the AI's true capabilities. During the training phase, tracking an Elo score would offer a clearer understanding of a model's progressive improvement over time, rather than relying on fixed win percentages. Within the multiplayer environment, these ratings could be utilized for fair matchmaking, dynamically pairing human players with other humans or AI agents of a similar skill level to ensure balanced gameplay.

# Conclusions

This thesis investigated the application of Reinforcement Learning to the stochastic, partially observable, and highly complex multi-agent environment of *Settlers of Catan*. By restricting the environment to a two-player competitive setting, the research successfully isolated and evaluated the core challenges of spatial expansion, long-term planning, and resource management.

The experimental results demonstrated that standard policy optimization (Pure-PPO) from a random initialization is fundamentally insufficient for this domain, hitting a strict performance ceiling of approximately 10% due to severe temporal credit assignment issues. Introducing Expert-Guided PPO (EG-PPO) successfully mitigated this bottleneck; by providing dense, step-by-step heuristic feedback, the agent achieved baseline competence and scaled its win rate beyond 30%. However, to achieve robust performance, this work demonstrated that expert knowledge must ultimately be decoupled from the reinforcement learning objective. By introducing a two-phase Behavioral Cloning pipeline (BC-PPO), where the agent was pre-trained on expert demonstrations before undergoing unconstrained PPO fine-tuning, the agent overcame previous limitations, achieving a peak win rate of 69.6% against a heuristic baseline.

The most critical and original finding of this research, however, is the paradox of teacher quality in Imitation Learning. The experiments proved that pre-training on a strategically superior multi-ply search agent (Alpha-Beta) paradoxically resulted in worse final performance than training on a simpler 1-ply heuristic. Because the neural network operates as a 1-ply reactive function approximator, it suffered from “unobserved intent” when attempting to mimic the deeper, hidden search tree of the Alpha-Beta agent. This highlights a vital conclusion for future reinforcement learning applications: architectural compatibility between the teacher’s decision-making process and the student’s observational capacity is fundamentally more critical than the absolute strategic strength of the demonstrator.

# Acknowledgements

I would like to express my sincere gratitude to the EuroHPC Joint Undertaking and the Vega high-performance computing cluster teams. Their generous provision of free computational resources was absolutely vital, making the large-scale experiments in this thesis possible.

A huge thank you goes to the developers of the Catanatron environment. Having access to such a robust, open-source simulation and Gymnasium interface was foundational to this work. I am especially grateful for their responsiveness and willingness to answer my questions during the development of this project.

# Bibliography

- [Cat95] Catan GmbH. Catan – the strategy board game. <https://www.catan.com>, 1995. Accessed: 2026-03-05.
- [Col23a] B. Collazo. Catanatron: Reinforcement Learning Environment for Settlers of Catan. <https://github.com/bcollazo/catanatron>, 2023.
- [Col23b] B. Collazo. Making catanatron stronger. <https://docs.catanatron.com/advanced/making-catanatron-stronger>, 2023.
- [Cul24] Cullback. Canopy: A catan bot. <https://github.com/cullback/canopy>, 2024.
- [Doc26] Docker Inc. Docker documentation, 2026. <https://docs.docker.com/>, Accessed: 2026-05-19.
- [Elo78] Arpad E. Elo. *The Rating of Chessplayers, Past and Present*. Arco Pub., New York, 1978.
- [JBS15] Michael B. Jones, John Bradley, and Nat Sakimura. JSON Web Token (JWT). RFC 7519, RFC Editor, May 2015. <https://www.rfc-editor.org/rfc/rfc7519.txt>.
- [KLM96] Leslie Pack Kaelbling, Michael L. Littman, and Andrew W. Moore. Reinforcement learning: A survey. *Journal of Artificial Intelligence Research*, 4:237–285, 1996.
- [M<sup>+</sup>15] Volodymyr Mnih et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.
- [MFPS06] A. McGovern, A.G. Fagg, N.J. Pintar, and R.S. Sutton. Policy gradient vs. value function approximation: A reinforcement learning shootout. Technical report, University of Massachusetts Amherst, 2006.
- [Pfe04] Michael Pfeiffer. Reinforcement learning of strategies for Settlers of Catan. In *Proceedings of the International Conference on Computer Games: Artificial Intelligence, Design and Education*, pages 384–388, 2004.
- [RHG<sup>+</sup>21] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268), 2021.

- [S<sup>+</sup>18] David Silver et al. A general reinforcement learning algorithm that masters chess, shogi and go through self-play. *Science*, 362:1140–1144, 2018.
- [SB18] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.
- [SCS10] István Szita, Guillaume Chaslot, and Pieter Spronck. Monte-carlo tree search in settlers of catan. In *Advances in Computer Games*, pages 21–32. Springer Berlin Heidelberg, 2010.
- [SHM<sup>+</sup>16] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.
- [SN26] Turculet Sorin-Nicolae. Imitation-Initialized Reinforcement Learning for Strategic Decision-Making in Settlers of Catan. *Studia Babeş-Bolyai Informatica*, page submitted, 2026.
- [VSP<sup>+</sup>17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30:5998–6008, 2017.
- [WMB23] Matthew Whelan, Nitin Maddi, and Sidhardh Burre. Reinforcement Learning in Catan, 2023. <https://sidhardhburre.com/CS6501RL/final.pdf>.